arXiv:2611.14844v1 [cs.AI] 10 Nov 2026

AUTOASSUMPTION: SEARCHING OVER ASSUMPTIONS DISCOVERS ML-SYSTEMS ALGORITHMS

Igor Sadalski [1] **Minghao Guo** [1] **Benjamin Jones** [1] **Wojciech Matusik** [1]

ABSTRACT

Many advances in ML systems came from breaking an assumption that earlier methods took for granted: speculative decoding dropped the assumption that a model needs one forward pass per token. Yet algorithms rarely state their assumptions, so experts and discovery harnesses search over programs, where one edit changes several design choices and gains are difficult to attribute. Searching over the assumptions themselves remains largely unexplored. Writing algorithms as proofs makes their assumptions explicit and turns algorithm design into a search over a small, structured space that an LLM can explore and a proof checker can audit. We formalize 89 algorithms from KV-cache eviction, weight pruning and speculative decoding as proof graphs with explicit assumptions, checked in Lean. Our harness, AutoAssumption, relaxes one assumption of a published method, re-derives and re-proves it, and generates its code. In KV-cache eviction and weight pruning, this search improved on every published algorithm we ran and produced state-of-the-art algorithms. The eviction algorithm has the highest RULER-4K score of the published methods we ran, and the pruning algorithm the lowest perplexity of thirteen on four LLaMA models. At equal budget, the harness scores within a point of OpenEvolve, an open-source implementation of AlphaEvolve, with a lower standard deviation across repeats. Future work will formalize more fields and combine them through the principal assumptions they share.

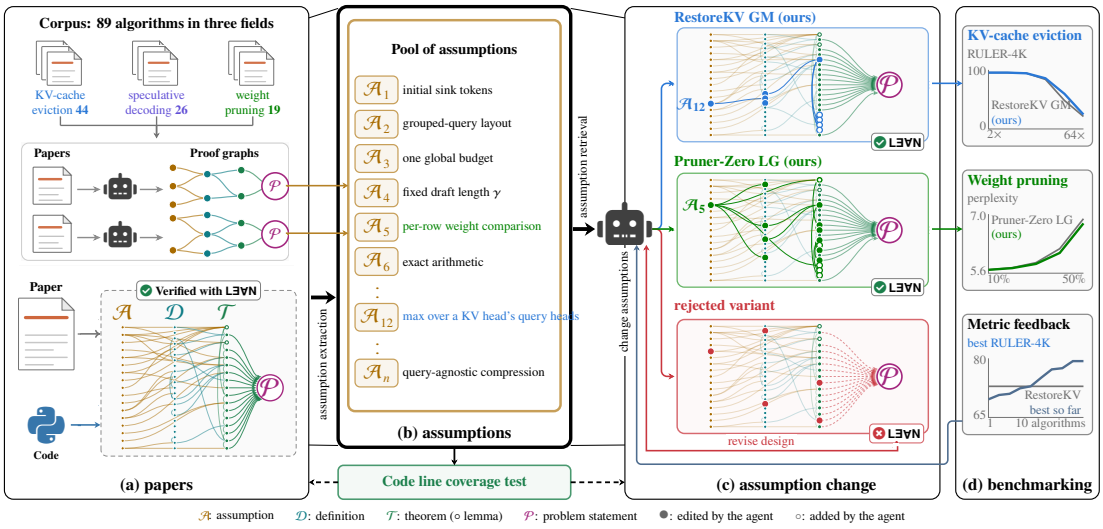

Figure 1. **AutoAssumption turns published algorithms into Lean-checked proofs and discovers new ones by changing one assumption at a time.** (a) An agent writes each published algorithm as a proof graph of assumptions, definitions and theorems, checked in Lean. (b) The assumptions of all proofs form one pool. (c) A variant changes one assumption of its parent and re-proves only the statements that depend on it; Lean rejects a proof that fails (dashed red). (d) Variants that pass are benchmarked, here on RULER-4K and WikiText-2 perplexity, and the scores guide the next change.

[1]MIT CSAIL, Cambridge, MA, USA. Correspondence to: Igor Sadalski <igors14@csail.mit.edu>.

Preprint.

1 INTRODUCTION

Many advances in ML systems came from overturning an assumption that earlier methods took for granted. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) dropped the assumption that every token needs its own pass of the

large model, and PagedAttention (Kwon et al., 2023) the assumption that a request's KV cache is contiguous. In KV-cache eviction, five of the seven records after H2O (Zhang et al., 2023) introduced a new assumption (Figure 2). Each such advance is easy to explain afterwards but was difficult to anticipate. We therefore hypothesize that assumptions, not code, are the productive unit of algorithm design.

Yet both experts and LLM harnesses search over programs, where it is difficult to tell which assumption produced a gain. Discovery harnesses such as AlphaEvolve, OpenEvolve and AIDE (Novikov et al., 2025; Sharma, 2025; Jiang et al., 2025) and AI-scientist systems (Lu et al., 2024b) edit code, and a single edit typically touches several design decisions together. They work on programs because published algorithms seldom spell out their assumptions. A paper often states an assumption in a single sentence or leaves it in its code, e.g., whether one KV-cache budget spans all layers, or when a speculative decoder stops drafting. Experts carry such assumptions as intuition, which makes it difficult to change them deliberately.

We propose to make the assumptions explicit by writing each algorithm as a logical proof. Starting from 89 algorithms with their papers and code, 44 in KV-cache eviction, 26 in speculative decoding and 19 in LLM weight pruning, an LLM proves that each solves its field's problem. We record the proof as a *proof graph*: its nodes are assumptions, definitions and theorems, and its edges are citations (Figures 3 and 4). A proof counts only if Lean (de Moura & Ullrich, 2021) checks it completely, with no unproved statements.

Once we have a proof, we can change one assumption and derive a new proof and a new algorithm. Changing one assumption of a proof graph points to the theorems that cite it, and thus to the parts of the algorithm that must change; re-deriving them yields a new algorithm with its own proof. For example, SVIP (Zhang et al., 2025) breaks the fixed draft length of speculative sampling, and its graph shows that only the cost claim depends on it. Doing this by hand is laborious, and these fields publish new algorithms faster than anyone can re-derive them (Appendix F).

We automate this process with our harness, AutoAssumption, which formalizes published algorithms and changes their assumptions (Figures 1 and 5). It identifies the problem a field's algorithms share, starting from the field's benchmark. It writes each algorithm's proof graph from its paper and code, checks it in Lean together with a coverage test that maps every line of code to a statement of the proof, and pools the assumptions. It then relaxes one assumption of a parent, re-derives and re-proves the dependent theorems, and generates the code before any benchmark run. We ask two questions about the harness: can it discover new performant algorithms, and how does it compare with other LLM harnesses?

In two fields, AutoAssumption improves on every published algorithm we ran, and it performs similarly to other harnesses. It found a higher-scoring variant for each of the 26 published algorithms it started from. RestoreKV group mean (ours) (RestoreKV GM) outperforms every published KV-cache eviction algorithm we ran on RULER-4K, averaged over $2\times$–$64\times$ compression and two models (Figure 7A). It outperforms its parent by up to 12.0 points at high compression (Figure 7B). Pruner-Zero layer group (ours) (Pruner-Zero LG) adds less perplexity over the dense model than any of the eleven published pruners we tested, at every sparsity on LLaMA-7B (tied with its parent at 10 %) and on every model at 50 % (Figure 7E and Table 3, bottom). SVIP lookup fallback (ours) (SVIP LK) is 4 % faster than SVIP at batch size 1 (Figure 7G) while keeping its exactness proof. With the same model and budget, the harness nearly matches OpenEvolve and scores 0.8 points above AIDE, and its results vary less across repeats (Section 4.2).

Searching over assumptions thus yields strong algorithms whose every change is interpretable and checked by proof. As next steps, we plan to formalize more fields, look for principal assumptions shared across them and unify the fields with this method.

Our contributions are:

1. **Two new state-of-the-art algorithms for KV-cache eviction and LLM weight pruning, discovered by our method.** The eviction algorithm scores 75.7 on RULER-4K, averaged over two models and $2\times$–$64\times$ compression, against 74.1 for the best published algorithm we ran. Under the SparseGPT and Wanda protocol, the pruning algorithm reaches a WikiText-2 perplexity of 6.78 on LLaMA-7B at 50 % sparsity, against 6.89, and stays best on four LLaMA models (Section 4). A third variant is a competitive speculative decoder.
2. **A blueprint for assumption-based algorithm search, with performance competitive with code-level harnesses.** AutoAssumption relaxes one assumption of a parent, re-derives only what depends on it, and checks the new proof in Lean before any benchmark run. With the same model and budget, it scores 0.4 points below OpenEvolve and 0.8 points above AIDE, with a lower standard deviation (SD) across repeats (0.75, against 1.39 and 4.09) (Section 4.2, Figure 8).
3. **Reformulation of 89 ML-systems algorithms across three fields.** We formalize 89 algorithms as proof graphs and merge their assumptions into one pool that points back to every paper.

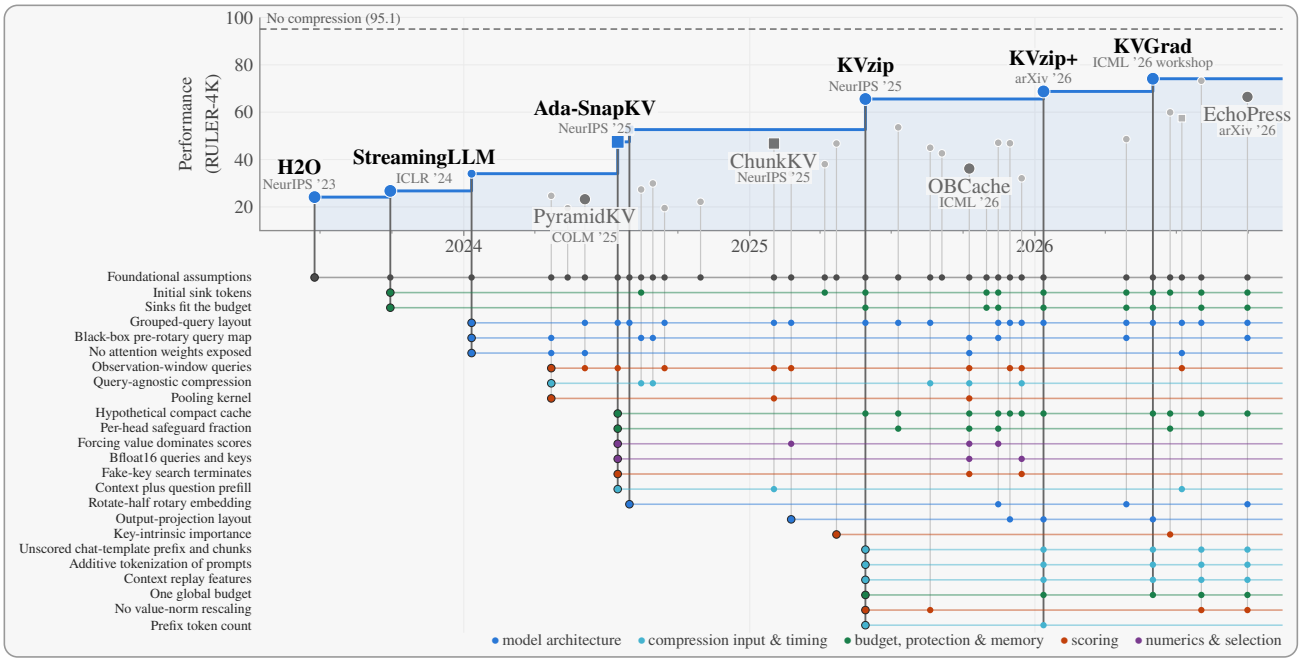

Figure 2. **Records in KV-cache eviction coincide with new assumptions.** Top: mean RULER-4K score of published algorithms by release date (KVPress, Llama-3.1-8B-Instruct and Qwen3-8B, 2×–64× compression, subsampled questions). The step line is the best score so far, bold labels name records, and squares mark query-aware algorithms. Bottom: one row per assumption shared by at least two algorithms; a dot marks an algorithm that makes it, and a ring the first algorithm to do so. The Foundational row holds the assumptions of H2O.

2 METHOD

We write each algorithm as a proof, change one assumption at a time and automate both with an LLM harness. Section 2.1 writes a published algorithm as a *proof graph*, a proof that it solves a formally stated problem with its assumptions explicit. Section 2.2 describes the LLM harness that automates the formalization and the derivation of variants over 89 algorithms in three fields (Appendix F).

2.1 Algorithms from three ML-systems fields can be formalized as proof graphs

We cast every algorithm of a field as a *proof graph*. A proof graph is a directed acyclic graph whose nodes are assumptions $\mathcal{A}$, definitions $\mathcal{D}$ and theorems $\mathcal{T}$, whose edges are citations, and which ends in the problem statement $\mathcal{P}$. The assumptions are what the argument takes as given, the definitions are what the program computes, and the theorems derive from both that the program solves $\mathcal{P}$. Figure 3 shows the proof graph of speculative sampling (SpS) (Leviathan et al., 2023; Chen et al., 2023). In SpS, a small drafter M_q proposes γ tokens and the large target M_p scores them in one forward pass; an accept-or-resample rule keeps the output distribution exactly that of M_p. Among its assumptions, the target and the drafter are distributions over the same finite vocabulary (Assumptions $\mathcal{A}_1$ and $\mathcal{A}_2$). Among its

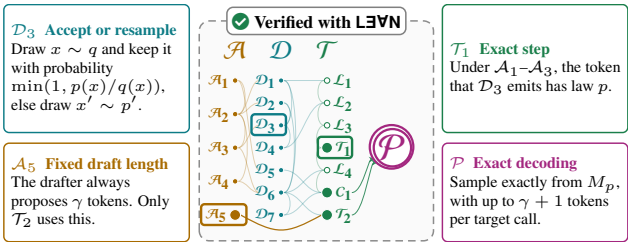

Figure 3. **The exactness proof of speculative sampling (Leviathan et al., 2023) can be represented as a proof graph.** The graph contains every node of Appendix H. Its columns hold the assumptions $\mathcal{A}$, the definitions $\mathcal{D}$ and the theorems $\mathcal{T}$; open discs are lemmas $\mathcal{L}$. Each call-out paraphrases the highlighted node of the same colour, and the badge marks that the proof compiles in Lean.

definitions is one accept-or-resample step (Definition $\mathcal{D}_3$). From these, it proves that the emitted token has the target's distribution (Theorem $\mathcal{T}_1$) and derives how many tokens a step emits in expectation (Theorem $\mathcal{T}_2$). Appendix H states and proves every node.

The problem statement is the algorithm's *contract*, and the theorems prove it. For speculative sampling, the contract states the output distribution and the number of tokens one target pass emits. A benchmark measures performance metrics, such as the speedup on Spec-Bench.

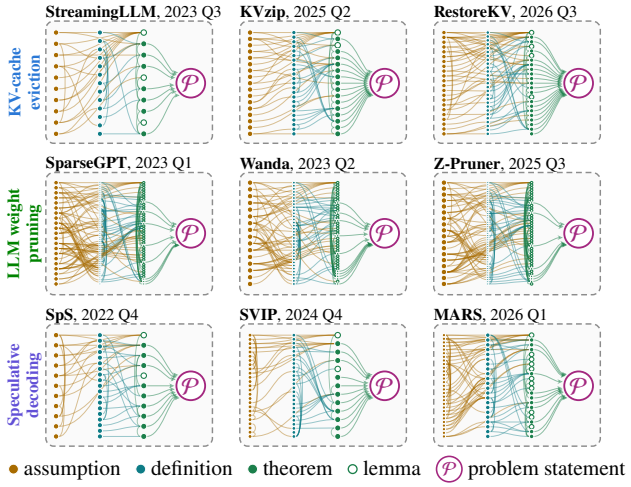

Figure 4. **Algorithms from three fields of ML systems can be written as proof graphs that answer the problem $\mathcal{P}$ shared within each field.** Each card is one algorithm's graph, drawn as in Figure 3; a line takes the colour of the node at its left end, and $\mathcal{P}$ is the problem of the card's field. The quarter after each name is the algorithm's first public release. Rows, left to right: StreamingLLM (Xiao et al., 2024), KVzip (Kim et al., 2025), RestoreKV (Baek et al., 2026); SparseGPT (Frantar & Alistarh, 2023), Wanda (Sun et al., 2024), Z-Pruner (Bhuiyan et al., 2025); SpS (Leviathan et al., 2023), SVIP (Zhang et al., 2025), MARS (Song et al., 2026). Appendix F lists every algorithm we analyze.

Proof graphs apply across fields. Figure 4 shows three speculative decoders, SpS, SVIP and MARS, next to three KV-cache eviction algorithms and three LLM weight-pruning algorithms.

2.2 Variant proofs can be automated with an LLM harness

Our harness automates the method in three stages (Appendix A.2). It identifies and formalizes the central problem, formalizes the existing solutions, and changes the assumptions of those solutions to generate variants.

Stage 1: identifying the problem. Stage 1 collects a field's reproducible algorithms from its benchmark and states one problem they all solve. We trace citation networks from the core evaluation of a field, i.e., its benchmark (Appendix A.2.1, Figure 10). We take the papers of the algorithms that the benchmark already implements, then the papers that cite them, and a person keeps the algorithms that are relevant and can run on the benchmark. We rewrite each kept algorithm in the benchmark as one self-contained Python file. The file must reproduce the benchmark's reported scores for an algorithm that the benchmark implements, and some of the results of the algorithm's own paper for an algorithm we ported. An LLM then reads these algorithms and proposes one problem that all of them can be proved to solve. We keep the problem once a random

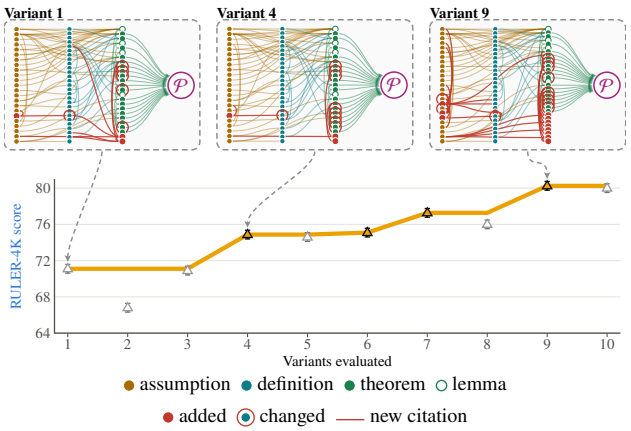

Figure 5. **A search chain in KV-cache eviction: each accepted variant changes one assumption of the current best and becomes the next parent.** The best of the three repeats of Figure 8, starting from RestoreKV, on a fixed draw of 200 RULER-4K questions. Above: the proof graphs of variants 1, 4 and 9. The line is the best variant score so far (RestoreKV itself, 72.9, is not plotted); filled triangles mark variants that set a new best and became the next parent, hollow ones variants that did not. Error bars (± 1 SD over question bootstraps, about 0.5; Appendix G.1) barely extend beyond the markers.

sample of algorithms can be formalized on it, and ask for a new formulation otherwise. A shared problem makes the assumptions of different algorithms comparable. For speculative decoding, the benchmark is Spec-Bench (Xia et al., 2024), and five of our algorithms are decoders it already implements. We ported 18 more and added 3 baselines, for 26 algorithms in all (Appendix F).

Stage 2: formalizing existing solutions. Two agents turn each algorithm into a Lean-checked proof, each following a fixed prompt (prompts in Appendix D, workflow in Appendix A.2.2 and Figure 11). The first agent receives the algorithm's folder, with its code, scores and paper (Appendix D.1). It confirms by hash that the code is the version that produced the scores, and writes the proof document from the code. We treat the code as the source of truth for what is defined, and use the paper as a guide to which assumptions to state. The document has one definition per logic line and ends in a main theorem that the code meets the problem's contract. A coverage test checks that the document covers every logic line of the code. The second agent transcribes the document into Lean (Appendix D.2). Its file states the assumptions, the definitions and the main theorem, and proves the theorem. A proof counts only if Lean checks it completely, with no step left unproved. For speculative decoding, the harness formalized all 26 algorithms. Their proofs establish exactness for eight published decoders, five of which had claimed it without proof (Appendix I.1). We then merge the assumptions of all proofs into one pool.

Stage 3: changing assumptions. We derive variants one

assumption at a time (workflow in Appendix A.2.3 and Figure 12). An agent receives a parent's code, proof and scores, the pool, and a prompt that describes how to derive a variant and its proof from the parent's proof (Appendix D.3). It makes one move on one assumption, which either relaxes or strengthens it. A relaxation drops the assumption, weakens it or replaces it with another algorithm's answer from the pool. Ideally, the move comes with a *dial*, a parameter of the new assumption whose value recovers the parent. The agent writes the new assumption and its hypothesis before any code, and only then derives the definitions and the code from them. Before any GPU time, the variant must differ from every known algorithm and from the parent on some input, and, if it has a dial, reproduce the parent at the dial's recovering value. Its proof is the parent's, edited only within the *impact set* of the changed assumption, the statements that cite it directly or through other statements; the rest of the proof stays as it is. Changes are marked in red. We then benchmark the variant in the parent's setting, and its scores guide the next choice of parent and assumption. Iterating from the best algorithm so far gives a search chain. Figure 5 shows one such chain in KV-cache eviction.

3 Experimental Setup

Our experiments ask two questions:

Q1 Can the harness discover new performant algorithms?
Q2 How does it compare with other LLM harnesses?

For Q1, we run the harness in three fields and score what it finds against each field's published methods. The fields are KV-cache eviction, LLM weight pruning and speculative decoding, each scored on its own benchmark. Table 1 gives the main benchmark, model, metric and error bar of each field; this section adds the other settings, so that others can reproduce each result. For Q2, we start OpenEvolve, AIDE and our harness from the same parent algorithm, with the same model, evaluator and budget, and measure the best score each finds. Appendix A lists model checkpoints, software versions and hardware (Table 4), and Appendix G explains every error bar.

KV-cache eviction. KV-cache eviction is scored on RULER and LongBench in KVPress, up to $64\times$ compression, with timing on a dedicated H100. KVPress (NVIDIA, 2024) is NVIDIA's eviction library and public leaderboard; it implements many published algorithms, and we ported the others to it. RULER (Hsieh et al., 2024) (Table 1) is a synthetic long-context benchmark of retrieval, variable-tracking, aggregation and question-answering tasks. To test generalization, we also use RULER at 8K and 16K tokens and LongBench (Bai et al., 2024), 3,750 questions from 16 English tasks with contexts of 2.6K to 30K tokens. Algorithms evict after prefill, before the question. Most prior work stops at $16\times$ compression; we extend to the high-compression regime of $16\times$ to $64\times$. System metrics are time to first token (TTFT), including prefill and compression, and the peak GPU memory that PyTorch allocates. To profile them reliably, we give each job a whole H100 GPU with no other load (verified in every run), run it at batch size 1 after an untimed warm-up, and time it over seven passes.

LLM weight pruning. Weight pruning follows the SparseGPT and Wanda protocol on four LLaMA models, scored by perplexity relative to the dense model. The protocol is that of SparseGPT (Frantar & Alistarh, 2023) and Wanda (Sun et al., 2024) (Table 1); beyond LLaMA-7B, we prune LLaMA-13B and 30B (Touvron et al., 2023a) and LLaMA-2-7B (Touvron et al., 2023b). Every linear layer of the decoder is pruned once, without retraining, to 10–50 % unstructured sparsity, using calibration windows of 2,048 tokens from C4 (Raffel et al., 2020), in fp16 on one H100 GPU. As in Pruner-Zero's official code, Pruner-Zero and Pruner-Zero LG (ours) also take gradient statistics of the dense model on WikiText-2. WikiText-2 (Merity et al., 2017) is a language-modeling corpus of Good and Featured Wikipedia articles; we concatenate its test split (60 articles, about 245K words), cut it into non-overlapping 2,048-token windows and take the exponential of the mean per-token negative log-likelihood. We report this perplexity divided by the dense model's, so that 1 means no loss from pruning. Baselines are eleven published algorithms, re-implemented from their official code (Appendix A.1), and magnitude pruning (Han et al., 2015). Each configuration is one run with a fixed calibration seed.

Speculative decoding. Speculative decoding is scored on Spec-Bench with two target–drafter pairs, with paired timing on a dedicated H200. Spec-Bench (Xia et al., 2024) (Table 1) comes with reference implementations of some published decoders; we ported the others to it. It spans six generation tasks, from multi-turn conversation and translation to mathematical reasoning and retrieval-augmented generation. Besides the Vicuna pair, we run Llama-3.1-8B-Instruct as target with Llama-3.2-1B-Instruct (Grattafiori et al., 2024) as drafter. At $T = 0$, decoding is greedy and a correct decoder reproduces the target's output token for token; at $T > 0$, decoding samples with top-$k = 50$ and a correct decoder reproduces the target's output distribution. Besides the speedup, we report the mean number of accepted tokens per target pass (#MAT), throughput at $T = 0$ and $T = 1$, and greedy time per output token (TPOT) at batch sizes 1 to 32. For reliable timing, each run has a whole H200 GPU with no other load, paired runs start minutes apart on one machine, and each speedup divides by an autoregressive run of the same target and temperature (Appendix A.1).

Harness comparison. The harness comparison gives OpenEvolve, AIDE, Fable from scratch and our harness the

Table 1. **We evaluate our designs on established benchmarks in each field.** The benchmark, model, metric and error bars behind Figure 7; Section 3 gives the other settings, Appendices A and G the versions, hardware and error bars in full.

	KV-cache eviction	LLM weight pruning	Speculative decoding
Benchmark	KVPress RULER-4K (13 tasks, 6,500 questions)	WikiText-2 test perplexity, SparseGPT and Wanda protocol (128 C4 calibration windows)	Spec-Bench (480 questions, 1,024 new tokens)
Model	Llama-3.1-8B-Instruct (Grattafiori et al., 2024) and Qwen3-8B (Yang et al., 2025)	LLaMA-7B (Touvron et al., 2023a)	Vicuna-7B-v1.3 (Chiang et al., 2023) target with the vicuna-68m drafter
Main metric	RULER-4K score (mean task accuracy), averaged over compression ratios 2×–64× and over the two models	WikiText-2 perplexity divided by the dense model's, at 50 % unstructured sparsity	speedup over autoregressive decoding at batch size 1, at sampling temperatures $T = 0$ to 1
Error bars	±1 SD over 10,000 paired bootstrap resamples of the questions	±1 paired bootstrap SE over the 166 WikiText-2 windows	±1 SD over three paired timing runs

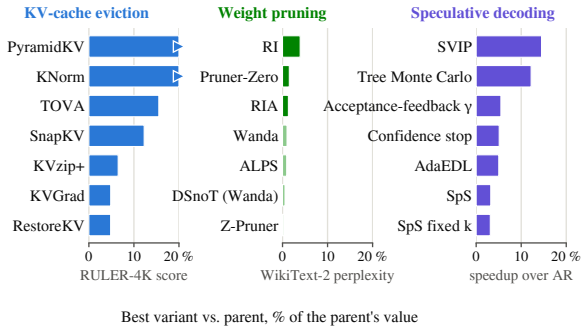

Figure 6. **For every parent algorithm, AutoAssumption generates a variant that scores higher than the parent on its field's search benchmark.** Across three fields, we selected 26 published algorithms as parents, and for each of them our harness found a better variant. Shown: the seven parents with the largest gain per field; Appendix Table 13 gives the results for the others. Triangles: gains beyond the axis; light bars: gains under 1 %.

same model, evaluator and budget. OpenEvolve (Sharma, 2025) and AIDE (Jiang et al., 2025) are code-level harnesses; *Fable from scratch*, a lower reference, writes algorithms from the task description alone. The model is Fable 5.1 at high reasoning effort, the budget ten benchmarked algorithms, and all except Fable from scratch start from RestoreKV. The benchmark is RULER-4K with Llama-3.1-8B-Instruct and Qwen3-8B, as above. Because our GPU runtime was limited, we score each algorithm on a fixed subset of 200 of its 6,500 questions. The metric is the best score found so far, where an algorithm's score is its mean accuracy over the two models and six compression ratios. We repeat each search three times and report the mean across repeats with the minimum and maximum as error bars. Appendix E describes how we ran OpenEvolve and AIDE.

4 Results

Our harness discovers state-of-the-art algorithms and matches other LLM harnesses. To answer Q1, whether the harness discovers new performant algorithms, we ran it from 26 published algorithms in three fields (11 in KV-cache eviction, 7 in weight pruning and 8 in speculative decoding), scored every variant on its field's search benchmark and kept the best one per parent. Every parent has a variant that scores higher (Figure 6), although the smallest gains lie within the benchmark noise. Three of these variants are new algorithms, each one assumption away from its parent (Table 2): RestoreKV GM and Pruner-Zero LG outperform every published algorithm we ran in KV-cache eviction and weight pruning (Figure 7A and D), and SVIP LK is faster than its parent SVIP at batch size 1 (Figure 7G). Section 4.1 tests them further against each field's published algorithms, on settings the search never saw and in cost. Section 4.2 answers Q2, whether the harness performs as well as other LLM harnesses, by comparing it with OpenEvolve and AIDE at the same budget; Appendix I adds what the proofs catch that benchmarks miss.

4.1 Q1: Can the harness discover new performant algorithms?

AutoAssumption sets the state of the art in two of the three tested ML-systems fields. In KV-cache eviction, RestoreKV GM (ours) scores 75.7 on RULER-4K, averaged over both models and six compression ratios, against 74.1 for KVGrad, the best published algorithm we ran (Figure 7A). It leads on each model separately (77.0 against 75.4 on Llama, 74.5 against 72.9 on Qwen; Appendix Table 5). The strong algorithms all stay near 95 at 2×–8×; the lead comes at 16× and 32×, up to 12.0 points over its parent (Figure 7B). In weight pruning, Pruner-Zero LG (ours) has the lowest perplexity of thirteen pruning algorithms we ran: on LLaMA-7B at 50 % sparsity it reaches 1.195 times the dense model's, against

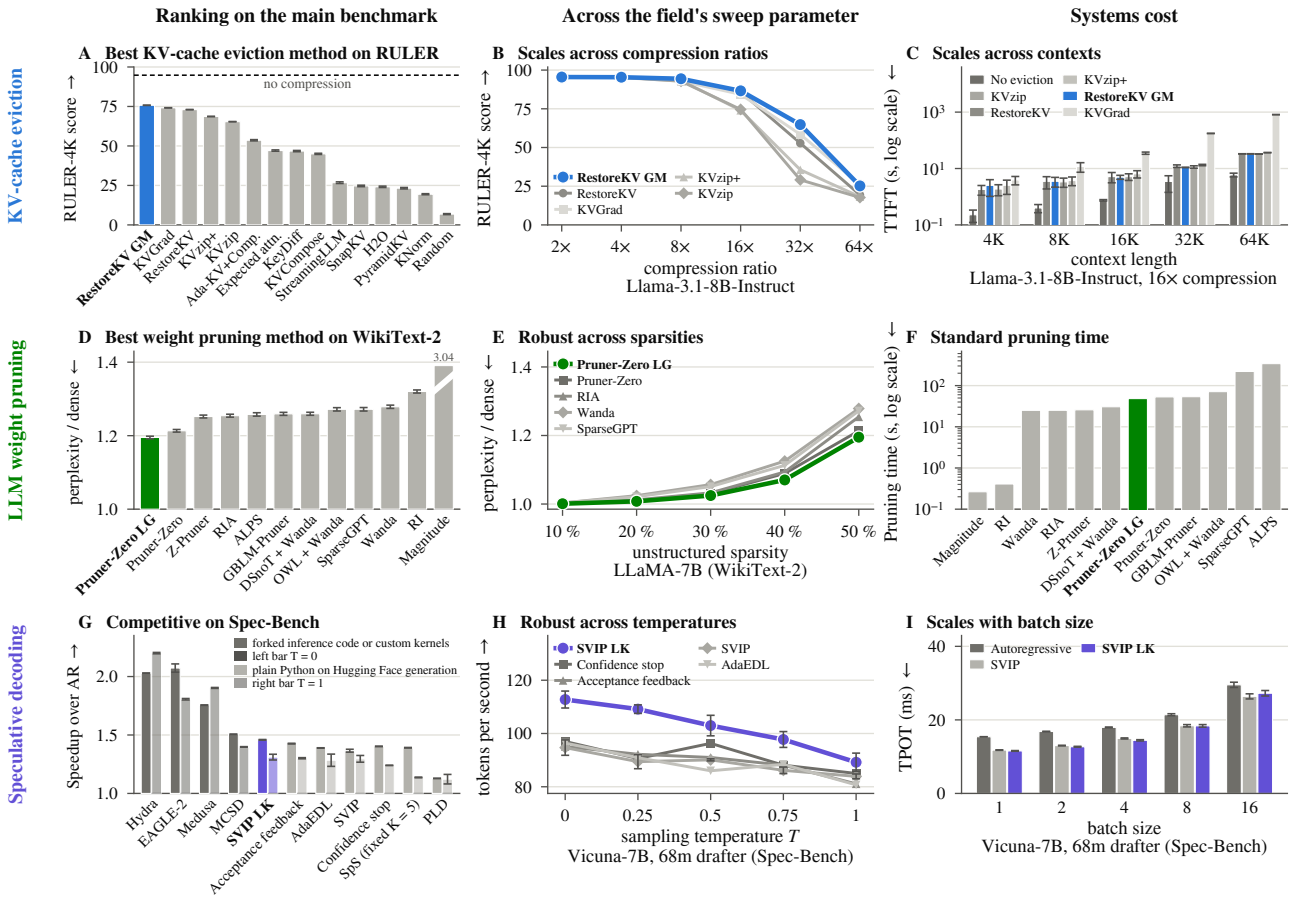

Figure 7. **Our three variants rank at or near the top of their fields, hold up across each field's sweep and remain practical to run.** AutoAssumption pushes the state of the art in KV-cache eviction and LLM weight pruning and creates a competitive speculative decoding algorithm (A, D, G). The three variants match or outperform the published algorithms shown at every point of the sweep each field reports, except at 64× KV-cache compression on Llama, where KVGrad leads by 0.8 points (B, E, H). Our variants take about as much time as their parents in all three fields (C, F, I). Error bars as in Table 1; in D, a paired bootstrap SE over the WikiText-2 windows.

Table 2. **We propose three new algorithms, each one assumption away from a published parent.** Names in the colour of their field; pseudocode in Appendix C.

Algorithm	Changed assumption	Code
RestoreKV GM	a cached pair is kept when the query heads that share its KV head attend to it on average, not when any single head does	Alg. 1
Pruner-Zero LG	weight scores are compared within a whole matrix instead of within each output row	Alg. 2
SVIP LK	when the entropy test ends a draft, tokens copied from an earlier occurrence in the prompt are appended instead	Alg. 3

1.213 for its parent Pruner-Zero (Dong et al., 2024) and 1.25 or more for every other published pruner (Figure 7D). In speculative decoding, SVIP LK (ours) is faster than SVIP (Zhang et al., 2025) at batch size 1, with a speedup of 1.46 against 1.37 at $T = 0$ and 1.31 against 1.30 at $T = 1$ (Figure 7G), and keeps its exactness proof. It is the fastest plain Python implementation we ran; only decoders with custom kernels or forked inference code are faster (Figure 7G). But does this performance generalize beyond the search setting?

The two best discovered variants generalize beyond the search setting. In KV-cache eviction, the search scored RestoreKV GM on 200 RULER-4K questions. On all 6,500 questions it improves its parent by 3.1 points on Llama-3.1-8B-Instruct and by 2.4 points on Qwen3-8B. The gain holds at longer contexts, by 3.1 and 3.3 points at 8K and 16K tokens, and on another dataset, by 1.5 points on LongBench (Bai et al., 2024) (Table 3, top). In LLM weight pruning, the search scored Pruner-Zero LG on LLaMA-7B at 50 % sparsity. It has the lowest perplexity of all thirteen pruning algorithms on LLaMA-7B, 13B and 30B, and on average adds 10 % less perplexity above the dense model than its parent, the best published pruner. It is also best on a different

Table 3. **The two best discovered variants hold beyond the search setting.** Top: RestoreKV GM (ours) against its parent, score averaged over 2×–64×; the search used 200 RULER-4K questions, and 8K and 16K contexts and LongBench are settings it never saw. Δ: variant minus parent, computed from unrounded means, ±1 SD from bootstrapping the questions. Bottom: WikiText-2 perplexity at 50 % sparsity divided by the dense model's, lower is better; the search used only LLaMA-7B. The ranking holds across all thirteen pruning algorithms we ran; the table shows a selection, and Appendix Table 9 gives every algorithm.

	RULER				LongBench
	Llama-3.1-8B			Qwen3-8B	Llama-3.1-8B
KV-cache eviction	4K	8K	16K	4K	2.6K–30K
RestoreKV	73.9	74.8	73.3	72.1	37.8
RestoreKV GM (ours)	**77.0**	**77.9**	**76.5**	**74.5**	**39.2**
Δ	+3.1 ±0.16	+3.1 ±0.17	+3.3 ±0.17	+2.4 ±0.17	+1.5 ±0.29

	LLaMA			LLaMA-2
Weight pruning	7B	13B	30B	7B
Pruner-Zero LG (ours)	**1.195**	**1.144**	**1.201**	**1.219**
Pruner-Zero	1.213	1.165	1.224	1.226
RIA	1.254	1.194	1.238	1.244
SparseGPT	1.272	1.219	1.297	1.283
Wanda	1.279	1.209	1.280	1.265

model, LLaMA-2-7B, by 3 % (Table 3, bottom), and at every sparsity from 10 % to 50 % on LLaMA-7B, tied with its parent at 10 % (Figure 7E).

The best variants remain practical, at costs close to their parents'. For each parent–variant pair we test the ratio of the variant's cost to its parent's for a difference. The time to first token of RestoreKV GM (ours) is 1.03 times RestoreKV's, pooled over contexts from 4K to 64K tokens, and not significantly different from it ($p = 0.69$; Figure 7C; Appendix Table 8). At 4K tokens on both models the difference is also not significant ($p = 0.39$), and its peak GPU memory is identical to its parent's. SVIP LK (ours) stays within 4% of SVIP's time per output token at every batch size up to 16 (Figure 7I); paired over questions and pooled over these batch sizes, it is 2.3% faster (90% CI 2.0–2.6%). At batch size 32, however, it is 11.7% slower (90% CI 10.6–12.7%). Pruner-Zero LG (ours) prunes LLaMA-7B in 48 s against 53 s for its parent (Figure 7F).

4.2 Q2: How does the harness compare with other LLM harnesses?

Searching over assumptions improves more consistently than searching over code. We compare with two code-level harnesses: OpenEvolve (Sharma, 2025), an open-source implementation of the prominent AlphaEvolve (Novikov et al., 2025), and AIDE (Jiang et al., 2025), a widely used tree search over programs. All three start from RestoreKV with the same model, evaluator and budget. The code-level harnesses gain most in their first steps: after two algorithms, OpenEvolve and AIDE reach 76.1 and 75.4 RULER points, against 70.7 for ours. We hypothesize that each of their edits changes several design choices at once, which allows a large early jump but gives little guidance for the next change; over the remaining eight algorithms they add only 3.6 and 3.1 points. Our harness changes one assumption per step and keeps rising, by 8.6 points over the same steps. After ten algorithms it reaches 79.3 points, against 79.7 for OpenEvolve and 78.5 for AIDE, while Fable from scratch converges lowest, at 71.4 (Figure 8, Table 12). Searching over assumptions thus matches searching over programs, with every change attributable to one assumption and a smaller spread across repeats (1.3 points after ten algorithms, against 2.5 and 7.1).

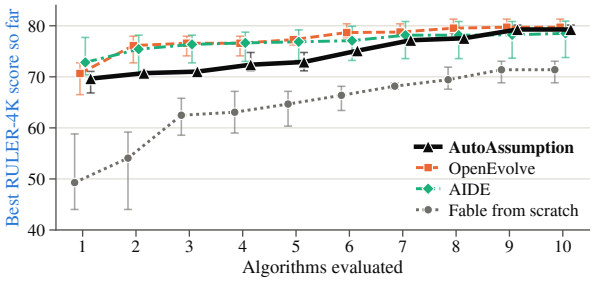

Figure 8. **AutoAssumption finishes level with the code-level LLM harnesses and improves more steadily, with a smaller spread across repeats.** OpenEvolve and AIDE jump early and then gain little; AutoAssumption, changing one assumption per step, rises at almost every step and ends at 79.3, against 79.7 and 78.5. All three start from RestoreKV; Fable from scratch has no parent. Scores average 200 RULER-4K questions, Llama-3.1-8B-Instruct and Qwen3-8B, and 2×–64× compression. Lines: mean of three repeats; error bars: minimum to maximum.

5 Related Work

Axiomatic design. The Axiomatic Atlas (Guo & Matusik, 2026) audits architectures against one fixed axiom set; we recover each algorithm's own assumptions. The Atlas states the requirements on a neural architecture as composable axioms over connectivity, operator contracts, depth stability and merges, and an audit returns a certificate or names the violated axiom, which implies a targeted repair. Our assumptions come from a Lean-checked proof of each algorithm, and a changed assumption dictates the re-proof and the code that the benchmark then scores.

Automated design search. Code-level harnesses evolve programs against a fixed evaluator (Romera-Paredes et al., 2024; Novikov et al., 2025; Sharma, 2025; Lange et al., 2026; Jiang et al., 2025), also for ML research (Lu et al., 2024a; Hu et al., 2025a; Lu et al., 2024b), systems research (Cheng et al., 2025b;a) and KV-cache eviction (Poudel et al., 2026; Yu & Chai, 2025; Park & Hwang, 2026). Their gains are difficult to attribute, and simple baselines often match

them (Gideoni et al., 2026; Oved et al., 2026). Our harness instead changes one stated assumption per step.

Assumptions and formal verification. Tracked assumptions go back to truth maintenance (de Kleer, 1986) and structured proofs (Lamport, 2012) and underlie formal blueprints (Massot, 2021; Pacalet et al., 2009). AssumptionMiner makes the assumptions behind LLM-generated code explicit (Wu, 2026); SampCert, AutoOPT, LegoNE, LLMLift and Algorithmist derive algorithms together with proofs (de Medeiros et al., 2025; Kim et al., 2026; Li et al., 2026; Bhatia et al., 2024; Kulkarni, 2026); and ProofLoom formalizes published algorithms in Lean (Wang et al., 2026). These works handle one algorithm at a time; we pool assumptions across a field and, as in proof repair (Ringer et al., 2019), re-prove only what a change affects.

6 Limitations

Three limitations remain: human review, fields that resist formalization and non-canonical assumptions. First, the formalization needs a human reader. Lean checks each proof against the problem statement, not the statement against the field, so a person must confirm that the contract says what the field means. Second, a proof covers only what mathematics can represent. In ML systems, a proof can show that an eviction algorithm uses less memory or that a decoder samples the target's distribution exactly. Training dynamics, or the quality of an LLM's answers, depend on statistics of the data that we do not fully understand, so we cannot build a proof around them; only the benchmark measures them. Third, the assumptions are not canonical. The LLM chooses how to split an algorithm into assumptions, and some of them, such as how the GPU is structured, admit several levels of detail or stay implicit. A second formalization could state a different set that still proves the contract and suggests different variants. We have not measured how stable the pool is across repeated formalizations.

7 Conclusion

ML-systems algorithms have advanced by breaking assumptions, yet these usually stay implicit. Writing algorithms as proofs makes them explicit and turns algorithm design into a search over a small, structured space that an LLM can explore and a proof checker can audit. In two fields, KV-cache eviction and weight pruning, this search improved on every published algorithm we ran and produced state-of-the-art algorithms. Our results support the view that assumptions, not code, are the productive unit of algorithm design: they make each change interpretable, its effect attributable and its correctness checkable. As future work, we plan to formalize more fields and improve their algorithms. We then plan to look for principal assumptions shared across fields and to combine the fields. Together, these steps would formalize the discipline as a whole.

References

Agrawal, S., Jeon, W., and Lee, M. AdaEDL: Early draft stopping for speculative decoding of large language models via an entropy-based lower bound on token acceptance probability. In *Proceedings of The 4th NeurIPS Efficient Natural Language and Speech Processing Workshop (ENLSP)*, volume 262 of *PMLR*, pp. 355–369, 2024. arXiv:2410.18351.

Ankner, Z., Parthasarathy, R., Nrusimha, A., Rinard, C., Ragan-Kelley, J., and Brandon, W. Hydra: Sequentially-dependent draft heads for Medusa decoding. In *Conference on Language Modeling (COLM)*, 2024.

Baek, C., Shin, S., and Kong, K. RestoreKV: Recovering full-cache behavior under aggressive query-agnostic KV cache eviction. arXiv:2608.01247, 2026.

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., and Li, J. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3119–3137, 2024.

Bhatia, S., Qiu, J., Hasabnis, N., Seshia, S. A., and Cheung, A. Verified code transpilation with LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.03003.

Bhuiyan, S. B., Adib, M. S. H., Bhuiyan, M. A., Kabir, M. R., Farazi, M., Rahman, S., and Mohammed, N. Z-Pruner: Post-training pruning of large language models for efficiency without retraining. In *2025 IEEE/ACS 22nd International Conference on Computer Systems and Applications (AICCSA)*, pp. 775–782, 2025. arXiv:2508.15828.

Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pp. 5209–5235, 2024.

Cai, Z., Zhang, Y., Gao, B., Liu, Y., Li, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Hu, J., and Xiao, W. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. In *Second Conference on Language Modeling (COLM)*, 2025. arXiv:2406.02069.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model

decoding with speculative sampling. arXiv:2302.01318, 2023.

Cheng, A., Liu, S., Pan, M., Li, Z., Agarwal, S., Cemri, M., Wang, B., Krentsel, A., Xia, T., Park, J., Yang, S., Chen, J., Agrawal, L., Naren, A., Li, S., Ma, R., Desai, A., Xing, J., Sen, K., Zaharia, M., and Stoica, I. Let the barbarians in: How AI can accelerate systems performance research. arXiv:2512.14806, 2025a.

Cheng, A., Liu, S., Pan, M., Li, Z., Wang, B., Krentsel, A., Xia, T., Cemri, M., Park, J., Yang, S., Chen, J., Agrawal, L., Desai, A., Xing, J., Sen, K., Zaharia, M., and Stoica, I. Barbarians at the gate: How AI is upending systems research. arXiv:2510.06189, 2025b.

Chiang, W.-L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Zhuang, Y., Gonzalez, J. E., Stoica, I., and Xing, E. P. Vicuna: An open-source chatbot impressing GPT-4 with 90%* ChatGPT quality. LMSYS blog post, https://lmsys.org/blog/2023-03-30-vicuna/, March 2023.

Das, R. J., Ranjan, M., Sun, M., Ma, L., and Shen, Z. Beyond size: How gradients shape pruning decisions in large language models. arXiv:2311.04902, 2023.

de Kleer, J. An assumption-based TMS. *Artificial Intelligence*, 28(2):127–162, 1986.

de Medeiros, M., Naveed, M., Lepoint, T., Kahsai, T., Ravitch, T., Zetzsche, S., Joshi, A., Tassarotti, J., Albarghouthi, A., and Tristan, J.-B. Verified foundations for differential privacy. *Proceedings of the ACM on Programming Languages*, 9(PLDI), 2025. doi: 10.1145/3729294.

de Moura, L. and Ullrich, S. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pp. 625–635. Springer, 2021.

Devoto, A., Zhao, Y., Scardapane, S., and Minervini, P. A simple and effective L_2 norm-based strategy for KV cache compression. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. arXiv:2406.11430.

Devoto, A., Jeblick, M., and Jégou, S. Expected attention: KV cache compression by estimating attention from future queries distribution. arXiv:2510.00636, 2025.

Dong, P., Li, L., Tang, Z., Liu, X., Pan, X., Wang, Q., and Chu, X. Pruner-Zero: Evolving symbolic pruning metric from scratch for large language models. In *International Conference on Machine Learning (ICML)*, 2024.

Feng, Y., Lv, J., Cao, Y., Xie, X., and Zhou, S. K. Ada-KV: Optimizing KV cache eviction by adaptive budget allocation for efficient LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2407.11550.

Feng, Y., Guo, H., Lv, J., Zhou, S. K., and Xie, X. DefensiveKV: Taming the fragility of KV cache eviction in LLM inference. In *International Conference on Learning Representations (ICLR)*, 2026.

Frantar, E. and Alistarh, D. SparseGPT: Massive language models can be accurately pruned in one-shot. In *International Conference on Machine Learning (ICML)*, 2023.

Fu, Y., Bailis, P., Stoica, I., and Zhang, H. Break the sequential dependency of LLM inference using lookahead decoding. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pp. 14060–14079, 2024.

Gante, J. Assisted generation: A new direction toward low-latency text generation. Hugging Face blog, https://huggingface.co/blog/assisted-generation, 2023.

Gideoni, Y., Risi, S., and Gal, Y. Simple baselines are competitive with code evolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. arXiv:2602.16805.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Roziere, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Wyatt, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Guzmán, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Thattai, G., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I., Misra, I., Evtimov, I., Zhang, J., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., van der Linde, J., Billock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J., Alwala, K. V., Prasad, K., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., El-Arini, K., Iyer, K., Malik, K., Chiu, K., Bhalla, K., Lakhotia, K., Rantala-Yeary, L., van der Maaten, L., Chen, L., Tan, L., Jenkins, L., Martin, L., Madaan, L., Malo, L., Blecher, L., Landzaat, L., Oliveira, L. d., Muzzi, M., Pasupuleti, M., Singh,

M., Paluri, M., Kardas, M., Tsimpoukelli, M., Oldham, M., Rita, M., Pavlova, M., Kambadur, M., Lewis, M., Si, M., Singh, M. K., Hassan, M., Goyal, N., Torabi, N., Bashlykov, N., Bogoychev, N., Chatterji, N., Zhang, N., Duchenne, O., Çelebi, O., Alrassy, P., Zhang, P., Li, P., Vasic, P., Weng, P., Bhargava, P., Dubal, P., Krishnan, P., Koura, P. S., Xu, P., He, Q., Dong, Q., Srinivasan, R., Ganapathy, R., Calderer, R., Cabral, R. S., Stojnic, R., Raileanu, R., Maheswari, R., Girdhar, R., Patel, R., Sauvestre, R., Polidoro, R., Sumbaly, R., Taylor, R., Silva, R., Hou, R., Wang, R., Hosseini, S., Chennabasappa, S., Singh, S., Bell, S., Kim, S. S., Edunov, S., Nie, S., Narang, S., Raparthy, S., Shen, S., Wan, S., Bhosale, S., Zhang, S., Vandenhende, S., Batra, S., Whitman, S., Sootla, S., Collot, S., Gururangan, S., Borodinsky, S., Herman, T., Fowler, T., Sheasha, T., Georgiou, T., Scialom, T., Speckbacher, T., Mihaylov, T., Xiao, T., Karn, U., Goswami, V., Gupta, V., Ramanathan, V., Kerkez, V., Gonguet, V., Do, V., Vogeti, V., Albiero, V., Petrovic, V., Chu, W., Xiong, W., Fu, W., Meers, W., Martinet, X., Wang, X., Wang, X., Tan, X. E., Xia, X., Xie, X., Jia, X., Wang, X., Goldschlag, Y., Gaur, Y., Babaei, Y., Wen, Y., Song, Y., Zhang, Y., Li, Y., Mao, Y., Coudert, Z. D., Yan, Z., Chen, Z., Papakipos, Z., Singh, A., Srivastava, A., Jain, A., Kelsey, A., Shajnfeld, A., Gangidi, A., Victoria, A., Goldstand, A., Menon, A., Sharma, A., Boesenberg, A., Baevski, A., Feinstein, A., Kallet, A., Sangani, A., Teo, A., Yunus, A., Lupu, A., Alvarado, A., Caples, A., Gu, A., Ho, A., Poulton, A., Ryan, A., Ramchandani, A., Dong, A., Franco, A., Goyal, A., Saraf, A., Chowdhury, A., Gabriel, A., Bharambe, A., Eisenman, A., Yazdan, A., James, B., Maurer, B., Leonhardi, B., Huang, B., Loyd, B., Paola, B. D., Paranjape, B., Liu, B., Wu, B., Ni, B., Hancock, B., Wasti, B., Spence, B., Stojkovic, B., Gamido, B., Montalvo, B., Parker, C., Burton, C., Mejia, C., Liu, C., Wang, C., Kim, C., Zhou, C., Hu, C., Chu, C.-H., Cai, C., Tindal, C., Feichtenhofer, C., Gao, C., Civin, D., Beaty, D., Kreymer, D., Li, D., Adkins, D., Xu, D., Testuggine, D., David, D., Parikh, D., Liskovich, D., Foss, D., Wang, D., Le, D., Holland, D., Dowling, E., Jamil, E., Montgomery, E., Presani, E., Hahn, E., Wood, E., Le, E.-T., Brinkman, E., Arcaute, E., Dunbar, E., Smothers, E., Sun, F., Kreuk, F., Tian, F., Kokkinos, F., Ozgenel, F., Caggioni, F., Kanayet, F., Seide, F., Florez, G. M., Schwarz, G., Badeer, G., Swee, G., Halpern, G., Herman, G., Sizov, G., Guangyi, Zhang, Lakshminarayanan, G., Inan, H., Shojanazeri, H., Zou, H., Wang, H., Zha, H., Habeeb, H., Rudolph, H., Suk, H., Aspegren, H., Goldman, H., Zhan, H., Damlaj, I., Molybog, I., Tufanov, I., Leontiadis, I., Veliche, I.-E., Gat, I., Weissman, J., Geboski, J., Kohli, J., Lam, J., Asher, J., Gaya, J.-B., Marcus, J., Tang, J., Chan, J., Zhen, J., Reizenstein, J., Teboul, J., Zhong, J., Jin, J., Yang, J., Cummings, J., Carvill, J., Shepard, J., McPhie, J., Torres, J., Ginsburg, J., Wang, J., Wu, K., U, K. H., Saxena, K., Khandelwal, K., Zand, K., Matosich, K., Veeraraghavan, K., Michelena, K., Li, K., Jagadeesh, K., Huang, K., Chawla, K., Huang, K., Chen, L., Garg, L., A, L., Silva, L., Bell, L., Zhang, L., Guo, L., Yu, L., Moshkovich, L., Wehrstedt, L., Khabsa, M., Avalani, M., Bhatt, M., Mankus, M., Hasson, M., Lennie, M., Reso, M., Groshev, M., Naumov, M., Lathi, M., Keneally, M., Liu, M., Seltzer, M. L., Valko, M., Restrepo, M., Patel, M., Vyatskov, M., Samvelyan, M., Clark, M., Macey, M., Wang, M., Hermoso, M. J., Metanat, M., Rastegari, M., Bansal, M., Santhanam, N., Parks, N., White, N., Bawa, N., Singhal, N., Egebo, N., Usunier, N., Mehta, N., Laptev, N. P., Dong, N., Cheng, N., Chernoguz, O., Hart, O., Salpekar, O., Kalinli, O., Kent, P., Parekh, P., Saab, P., Balaji, P., Rittner, P., Bontrager, P., Roux, P., Dollar, P., Zvyagina, P., Ratanchandani, P., Yuvraj, P., Liang, Q., Alao, R., Rodriguez, R., Ayub, R., Murthy, R., Nayani, R., Mitra, R., Parthasarathy, R., Li, R., Hogan, R., Battey, R., Wang, R., Howes, R., Rinott, R., Mehta, S., Siby, S., Bondu, S. J., Datta, S., Chugh, S., Hunt, S., Dhillon, S., Sidorov, S., Pan, S., Mahajan, S., Verma, S., Yamamoto, S., Ramaswamy, S., Lindsay, S., Lindsay, S., Feng, S., Lin, S., Zha, S. C., Patil, S., Shankar, S., Zhang, S., Zhang, S., Wang, S., Agarwal, S., Sajuyigbe, S., Chintala, S., Max, S., Chen, S., Kehoe, S., Satterfield, S., Govindaprasad, S., Gupta, S., Deng, S., Cho, S., Virk, S., Subramanian, S., Choudhury, S., Goldman, S., Remez, T., Glaser, T., Best, T., Koehler, T., Robinson, T., Li, T., Zhang, T., Matthews, T., Chou, T., Shaked, T., Vontimitta, V., Ajayi, V., Montanez, V., Mohan, V., Kumar, V. S., Mangla, V., Ionescu, V., Poenaru, V., Mihailescu, V. T., Ivanov, V., Li, W., Wang, W., Jiang, W., Bouaziz, W., Constable, W., Tang, X., Wu, X., Wang, X., Wu, X., Gao, X., Kleinman, Y., Chen, Y., Hu, Y., Jia, Y., Qi, Y., Li, Y., Zhang, Y., Zhang, Y., Adi, Y., Nam, Y., Yu, Wang, Zhao, Y., Hao, Y., Qian, Y., Li, Y., He, Y., Rait, Z., DeVito, Z., Rosnbrick, Z., Wen, Z., Yang, Z., Zhao, Z., and Ma, Z. The Llama 3 herd of models. arXiv:2407.21783, 2024.

Guo, M. and Matusik, W. Axiomatic Atlas: A prescriptive framework for neural architecture design. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *PMLR*, pp. 38131–38212, 2026.

Han, S., Pool, J., Tran, J., and Dally, W. J. Learning both weights and connections for efficient neural network. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.

He, Z., Zhong, Z., Cai, T., Lee, J. D., and He, D. REST: Retrieval-based speculative decoding. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human*

Language Technologies (Volume 1: Long Papers), pp. 1582–1595, 2024.

Hsieh, C.-P., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., and Ginsburg, B. RULER: What's the real context size of your long-context language models? In *First Conference on Language Modeling (COLM)*, 2024.

Hu, S., Lu, C., and Clune, J. Automated design of agentic systems. In *International Conference on Learning Representations (ICLR)*, 2025a. arXiv:2408.08435.

Hu, Y., Wang, K., Zhang, X., Zhang, F., Li, C., Chen, H., and Zhang, J. SAM decoding: Speculative decoding via suffix automaton. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, pp. 12187–12204, 2025b.

Jiang, Z., Schmidt, D., Srikanth, D., Xu, D., Kaplan, I., Jacenko, D., and Wu, Y. AIDE: AI-driven exploration in the space of code. arXiv:2502.13138, 2025.

Kim, H., Ryu, E. K., and Das Gupta, S. A domain-specific harness for end-to-end automation of optimization research. arXiv:2608.07407, 2026.

Kim, J.-H., Kim, J., Kwon, S., Lee, J. W., Yun, S., and Song, H. O. KVzip: Query-agnostic KV cache compression with context reconstruction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.23416.

Kulkarni, J. Early discoveries of Algorithmist I: Promise of provable algorithm synthesis at scale. arXiv:2603.22363, 2026.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

Lamport, L. How to write a 21st century proof. *Journal of Fixed Point Theory and Applications*, 11(1):43–63, 2012.

Lange, R. T., Imajuku, Y., and Cetin, E. ShinkaEvolve: Towards open-ended and sample-efficient program evolution. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.19349.

Lee, J., Park, S., Mo, S., Ahn, S., and Shin, J. Layer-adaptive sparsity for the magnitude-based pruning. In *International Conference on Learning Representations (ICLR)*, 2021.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pp. 19274–19286, 2023.

Li, H., Li, D., and Deng, X. Discovering expert-level Nash equilibrium algorithms with large language models. *Nature Communications*, 17:7168, 2026. doi: 10.1038/s41467-026-74003-1.

Li, Y., Niu, L., Zhang, X., Liu, K., Zhu, J., and Kang, Z. E-Sparse: Boosting the large language model inference through entropy-based N:M sparsity. arXiv:2310.15929, 2023.

Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024a.

Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 7421–7432, 2024b.

Lin, J., Geng, S., and Bourgeat, T. EchoPress: Query-agnostic KV cache pruning via virtual context reconstruction. arXiv:2610.00412, 2026.

Lu, C., Holt, S., Fanconi, C., Chan, A. J., Foerster, J., van der Schaar, M., and Lange, R. T. Discovering preference optimization algorithms with and for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024a.

Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., and Ha, D. The AI scientist: Towards fully automated open-ended scientific discovery. arXiv:2408.06292, 2024b.

Luo, X., Wang, Y., Zhu, Q., Zhang, Z., Zhang, X., Yang, Q., and Xu, D. Turning trash into treasure: Accelerating inference of large language models with token recycling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, pp. 6816–6831, 2025.

Mamou, J., Pereg, O., Gante, J., Tunstall, L., Korat, D., Timor, N., and Wasserblat, M. Faster assisted generation with dynamic speculation. Hugging Face blog, https://huggingface.co/blog/dynamic_speculation_lookahead, October 2024a.

Mamou, J., Pereg, O., Korat, D., Berchansky, M., Timor, N., Wasserblat, M., and Schwartz, R. Dynamic speculation lookahead accelerates speculative decoding of large language models. In *Proceedings of the 4th NeurIPS Efficient Natural Language and Speech Processing Workshop (ENLSP)*, volume 262 of *PMLR*, pp. 456–467, 2024b.

Massot, P. leanblueprint: a plasTeX plugin for writing blueprints of Lean projects. GitHub repository, https://github.com/PatrickMassot/leanblueprint, 2021. Accessed October 2026.

Meng, X., Behdin, K., Wang, H., and Mazumder, R. ALPS: Improved optimization for highly sparse one-shot pruning for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017.

Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J. R., Mehrabian, A., Kumar, M. P., See, A., Chaudhuri, S., Holland, G., Davies, A., Nowozin, S., Kohli, P., and Balog, M. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131, 2025.

NVIDIA. KVPress: LLM KV cache compression made easy. GitHub repository, https://github.com/NVIDIA/kvpress; leaderboard https://huggingface.co/spaces/nvidia/kvpress-leaderboard, 2024. Accessed October 2026.

Oved, T., Pony, R., Naparstek, O., and Barzelay, U. Evolution or illusion? Rethinking evaluation in LLM evolutionary search. arXiv:2609.19799, 2026.

Pacalet, A., Bertot, Y., and Pons, O. coq-dpdgraph: Build dependency graphs between Coq objects. GitHub repository, https://github.com/coq-community/coq-dpdgraph, 2009.

Park, J., Jones, D., Morse, M. J., Goel, R., Lee, M., and Lott, C. KeyDiff: Key similarity-based KV cache eviction for long-context LLM inference in resource-constrained environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, 2025.

Park, W. and Hwang, S.-w. TaskPress: Query-agnostic KV cache compression via task-guided pruning. arXiv:2608.03276, 2026.

Poudel, P., Wu, Y., Jha, S., and Liu, J. Discovering KV cache eviction policies via LLM-guided program evolution. arXiv:2608.14555, 2026.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21 (140):1–67, 2020.

Ringer, T., Palmskog, K., Sergey, I., Gligoric, M., and Tatlock, Z. QED at large: A survey of engineering of formally verified software. *Foundations and Trends in Programming Languages*, 5(2–3):102–281, 2019.

Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J. R., Ellenberg, J. S., Wang, P., Fawzi, O., Kohli, P., and Fawzi, A. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

Saxena, A. Prompt lookup decoding. https://github.com/apoorvumang/prompt-lookup-decoding/, November 2023.

Sengupta, A., Chaudhary, S., and Chakraborty, T. Value-guided KV compression for LLMs via approximated CUR decomposition. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, 2025.

Sharma, A. OpenEvolve: An open-source evolutionary coding agent. GitHub repository, https://github.com/algorithmicsuperintelligence/openevolve, 2025.

Song, J., Wang, X., Wang, H., Lei, X., Shi, T., Han, S., Yang, E., Chang, X.-W., and Ai, L. MARS: Unleashing the power of speculative decoding via margin-aware verification. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 8652–8663, 2026. doi: 10.18653/v1/2026.findings-acl.421. arXiv:2601.15498.

Sun, M., Liu, Z., Bair, A., and Kolter, J. Z. A simple and effective pruning approach for large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. LLaMA: Open and efficient foundation language models. arXiv:2302.13971, 2023a.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Canton Ferrer, C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M.-A., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R.,

Edunov, S., and Scialom, T. Llama 2: Open foundation and fine-tuned chat models. arXiv:2307.09288, 2023b.

Wang, F., Li, D., and Yuan, K. ProofLoom: Proof-obligation-driven theory construction for autoformalizing research-level stochastic optimization. arXiv:2609.34960, 2026.

Wu, J. AssumptionMiner: Extracting, tracing, and revising implicit assumptions in LLM code generation. arXiv:2607.22898, 2026.

Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., and Sui, Z. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 7655–7671, 2024.

Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report. arXiv:2505.09388, 2025.

Yang, J., Tang, C., Zhen, L., and Lv, J. Rethinking KV cache eviction via a unified information-theoretic objective. In *International Conference on Machine Learning (ICML)*, 2026.

Yang, S., Huang, S., Dai, X., and Chen, J. Multi-candidate speculative decoding. arXiv:2401.06706, 2024.

Yin, L., Wu, Y., Zhang, Z., Hsieh, C.-Y., Wang, Y., Jia, Y., Li, G., Jaiswal, A. K., Pechenizkiy, M., Liang, Y., Bendersky, M., Wang, Z., and Liu, S. Outlier weighed layerwise sparsity (OWL): A missing secret sauce for pruning LLMs to high sparsity. In *International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pp. 57101–57115, 2024.

Yu, B. and Chai, Y. EvolKV: Evolutionary KV cache compression for LLM inference. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 1673–1689, 2025.

Zhang, Y., Bai, H., Lin, H., Zhao, J., Hou, L., and Cannistraci, C. V. Plug-and-play: An efficient post-training pruning method for large language models. In *International Conference on Learning Representations (ICLR)*, 2024a.

Zhang, Y., Du, Y., Luo, G., Zhong, Y., Zhang, Z., Liu, S., and Ji, R. CaM: Cache merging for memory-efficient LLMs inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pp. 58840–58850, 2024b.

Zhang, Y., Zhao, L., Lin, M., Sun, Y., Yao, Y., Han, X., Tanner, J., Liu, S., and Ji, R. Dynamic sparse no training: Training-free fine-tuning for sparse LLMs. In *International Conference on Learning Representations (ICLR)*, 2024c.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., and Chen, B. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

Zhang, Z., Xu, J., Liang, T., Chen, X., He, Z., Wang, R., and Tu, Z. Draft model knows when to stop: Self-verification speculative decoding for long-form generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 16685–16697, 2025.

AI USAGE DISCLOSURE

Under the authors' direction, Claude Code (Anthropic) was used for experiment code, scheduling and evaluation scripts, figure and table scripts, and manuscript assistance. OpenAI Codex assisted this revision by auditing consistency, checking existing numerical artifacts, generating a matched comparison figure and editing the manuscript. The authors are responsible for reviewing the final text, code, citations and claims. LLM-generated algorithms, proof documents and code studied as experimental outputs are identified as such in the paper.

CONTENTS OF THE APPENDIX

A Reproducibility **18**

A.1 Reproducing external results . . . 18

A.2 Reproducing our harness . . . 19

B Results in numbers **24**

B.1 KV-cache eviction . . . 24

B.2 Speculative decoding . . . 26

B.3 LLM weight pruning . . . 26

B.4 Harness comparison . . . 27

B.5 Best variant of every parent . . . 28

C Algorithm sketches **30**

C.1 KV-cache eviction . . . 30

C.2 LLM weight pruning . . . 31

C.3 Speculative decoding . . . 32

D Prompts for models **33**

D.1 Designing proof files . . . 34

D.2 Designing Lean from proof files . . . 38

D.3 Designing variants . . . 40

E OpenEvolve and AIDE as run here **44**

F All algorithms **45**

G Uncertainty estimation **46**

G.1 KV-cache eviction: resampling the questions . . . 46

G.2 Speculative decoding and systems profiles: repeating the runs . . . 46

G.3 Design searches: repeating the search . . . 46

H The worked example in full **47**

H.1 The setting and the problem . . . 47

H.2 One position: accept or resample . . . 48

H.3 The emitted token has the target's law . . . 49

H.4 From one token to a block, and to the whole loop . . . 49

H.5 How many tokens one target pass buys . . . 50

I Mutation study **52**

I.1 Exactness proofs of published decoders . . . 52
I.2 Proof checks catch every injected bug, benchmarks only about half . . . 52

LIST OF FIGURES

1 AutoAssumption turns published algorithms into Lean-checked proofs and discovers new ones by changing one assumption at a time. . . . 1
2 Records in KV-cache eviction coincide with new assumptions. . . . 3
3 The exactness proof of speculative sampling (Leviathan et al., 2023) can be represented as a proof graph. 3
4 Algorithms from three fields of ML systems can be written as proof graphs that answer the problem $\mathcal{P}$ shared within each field. . . . 4
5 A search chain in KV-cache eviction: each accepted variant changes one assumption of the current best and becomes the next parent. . . . 4
6 For every parent algorithm, AutoAssumption generates a variant that scores higher than the parent on its field's search benchmark. . . . 6
7 Our three variants rank at or near the top of their fields, hold up across each field's sweep and remain practical to run. . . . 7
8 AutoAssumption finishes level with the code-level LLM harnesses and improves more steadily, with a smaller spread across repeats. . . . 8
9 Our runs reproduce the published leaderboards. . . . 19
10 Stage 1 selects a field's reproducible algorithms and finds one problem that all of them can be proved to solve. 21
11 Formalizing a published algorithm with two agents. . . . 22
12 Making one variant. . . . 23
13 KVGrad EA (ours) lies above every prior design at $16\times$–$64\times$ compression, where it gains 6.9 points on average over its parent KVGrad (3.7 points over $2\times$–$64\times$). . . . 24
3 The proof graph of this appendix, repeated from Figure 3. . . . 47
14 Lean flags every mutant; the benchmark flags about half. . . . 52

LIST OF TABLES

1 We evaluate our designs on established benchmarks in each field. . . . 6
2 We propose three new algorithms, each one assumption away from a published parent. . . . 7
3 The two best discovered variants hold beyond the search setting. . . . 8
4 GPU, NVIDIA driver, CPU, Python, PyTorch, libraries and precision of the machine behind each field's experiments (KV-cache eviction, weight pruning, speculative decoding). . . . 18
5 RULER-4K scores of the published algorithms named in Figure 13 and of RestoreKV GM (ours), per model and compression ratio. . . . 24
6 Full RULER-4K scores (6,500 questions) of KVzip, KVzip+, RestoreKV and RestoreKV GM (ours), per model and compression ratio. . . . 25
7 Scores of RestoreKV and RestoreKV GM (ours) at 8K and 16K contexts, on Qwen3-8B and on LongBench, per compression ratio. . . . 25
8 Time to first token of no eviction, KVGrad, RestoreKV, KVzip, KVzip+ and RestoreKV GM (ours) at 4K to 64K tokens. . . . 25

9 WikiText-2 test perplexity of eleven published pruners and Pruner-Zero LG (ours) on LLaMA 7B, 13B and 30B and LLaMA-2 7B at 50 % unstructured sparsity. 26

10 WikiText-2 test perplexity of eleven published pruners and Pruner-Zero LG (ours) on LLaMA-7B at 10, 20, 30 and 40 % unstructured sparsity. 27

11 Zero-shot accuracy on seven tasks of eleven published pruners and Pruner-Zero LG (ours) on LLaMA-7B at 50 % unstructured sparsity. 27

12 Running best RULER-4k score after 1 to 10 algorithms of Fable from scratch, AIDE, OpenEvolve and our harness (the numbers of Figure 8). 28

13 The best variant of each of the 26 parent algorithms on its field's search benchmark. 29

A REPRODUCIBILITY

This appendix gives what is needed to rerun our experiments. Appendix A.1 lists the benchmarks, models and machines of the three fields, and Appendix A.2 gives the steps by which our harness produces a design proof and a variant.

A.1 Reproducing external results

Each field ran on its own machine (Table 4), one job per whole GPU through a shared queue. Every score records its machine, software versions, run date and the SHA-256 of its code, and each algorithm is released with its proof, code and scores. Checkpoints and datasets come from the Hugging Face Hub (Spec-Bench from GitHub) at the revisions recorded with the results; Appendix E pins OpenEvolve and AIDE.

Table 4. **GPU, NVIDIA driver, CPU, Python, PyTorch, libraries and precision of the machine behind each field's experiments (KV-cache eviction, weight pruning, speculative decoding).** One GPU per job. Driver column: highest CUDA version the driver supports; brackets: CUDA runtime PyTorch was built against.

Experiments	GPU (one per job)	NVIDIA driver (CUDA)	CPU	Python	PyTorch (CUDA runtime)	Libraries	Precision
KV-cache eviction	NVIDIA H100 80GB HBM3	550.144.03 (CUDA 12.4)	2× AMD EPYC 9554, 128 cores	3.10.12	2.14.1+cu126 (12.6)	KVPress 0.5.5, Transformers 5.2.0	BF16
Weight pruning	NVIDIA H100 80GB HBM3	550.144.03 (CUDA 12.4)	2× AMD EPYC 9554, 128 cores	3.10.12	2.14.1+cu126 (12.6)	Transformers 5.2.0, Datasets 5.0.1, lm-eval 0.4.13	FP16
Speculative decoding	NVIDIA H200, 141 GB	595.58.03 (CUDA 13.2)	2× Intel Xeon Platinum 8592+, 256 threads	3.12.13	2.5.1+cu124 (12.4)	Spec-Bench (commit fd2c1cd7), Transformers 4.37.1	FP16

KV-cache eviction. Every algorithm runs as a press of KVPress (NVIDIA, 2024) 0.5.5 on one H100, evicting after prefill and before the question at compression ratios from $2\times$ to $64\times$. The models are Llama-3.1-8B-Instruct (`NousResearch/Meta-Llama-3.1-8B-Instruct`, with Meta's Llama-3.1 chat template) and Qwen3-8B (`Qwen/Qwen3-8B`), in BF16. RULER (Hsieh et al., 2024) (`simonjegou/ruler`) has 13 synthetic tasks at 4,096 tokens, 500 questions each, scored by the mean per-task accuracy. The searches used a nested 200-question subset (seed 42), so the 6,500-question scores of Table 6 are not held out.

The other KV-cache settings ran on two other machines. RULER at 8K and 16K tokens and the 16 English tasks of LongBench (Bai et al., 2024) (`Xnhyacinth/LongBench`, 3,750 questions) in Table 3 (top) ran on a second H100 machine (NVIDIA driver 580.178.04, CUDA 13.0, Intel Xeon Platinum 8481C, PyTorch 2.14.1+cu130, otherwise as the first row of Table 4). The 300-question scores of the prior algorithms in Figures 2 and 13 ran on the H200 machine with the KV-cache software (PyTorch 2.14.1+cu130). Our re-runs, the parent rows of Tables 6 and 3, agree with NVIDIA's KVPress leaderboard within 0.4 RULER points across all 41 points (Figure 9a).

Weight pruning. Pruning follows the protocol of SparseGPT and Wanda on LLaMA-7B, 13B and 30B (`huggyllama/llama-7b`, `-13b`, `-30b`) and LLaMA-2-7B (`NousResearch/Llama-2-7b-hf`) in FP16. Every linear layer of the decoder is pruned once, layer by layer, without retraining. Calibration draws 128 random 2,048-token windows (seed 0) from the first C4 training shard (`allenai/c4`, `en/c4-train.00000-of-01024`); PrunerZero computes its gradient statistics of the dense model on WikiText-2. The scores are WikiText-2 test perplexity (`wikitext-2-raw-v1`, 2,048-token windows) and, at 50 %, the mean zero-shot accuracy of lm-eval 0.4.13 on Wanda's seven tasks (BoolQ, RTE, HellaSwag, WinoGrande, ARC-easy, ARC-challenge, OpenBookQA).

The eleven published algorithms are single files ported from their official code, run in one harness on the KV-cache H100 machine (78 runs on the second H100 machine). Where their papers report a cell, our perplexities agree within 0.1. Each cell is one run; its error bar is a bootstrap SE over the 166 WikiText-2 windows.

Speculative decoding. Spec-Bench (Xia et al., 2024) (`hemingkx/Spec-Bench`, commit `fd2c1cd`) has 480 questions in six tasks. The target–drafter pairs are Vicuna-7B-v1.3 (`lmsys/vicuna-7b-v1.3`) with vicuna-

68m (`double7/vicuna-68m`) and Llama-3.1-8B-Instruct (`NousResearch/Meta-Llama-3.1-8B-Instruct`) with Llama-3.2-1B-Instruct (`unsloth/Llama-3.2-1B-Instruct`); the two Llama models share Meta's Llama-3.1 chat template and tokenizer. The native baselines use Spec-Bench's code with the Medusa (`FasterDecoding/medusa-vicuna-7b-v1.3`), EAGLE (`yuhuili/EAGLE-Vicuna-7B-v1.3`) and Hydra (`ankner/hydra-vicuna-7b-v1.3`) heads; SVIP follows its authors' code (`Geralt-Targaryen/SVIP`, commit `0cd912e`). Every algorithm runs in Spec-Bench's unmodified evaluation loop, which does the prompting, warm-up and timing, on all 480 questions, with at most 1,024 new tokens, in FP16 under Transformers 4.37.1.

SVIP keeps its authors' constants (stop drafting when $\sqrt{cH} > h$ with $c = 0.15$, $h = 0.3$; at most 40 draft tokens), and SVIP LK (ours) adds only PLD's prompt-lookup parameters (suffix n-grams up to 3, at most 10 lookup tokens). At $T > 0$, drafter and target sample with the same temperature and top-$k = 50$. Each #MAT and speedup cell, at five temperatures, is the mean of three paired runs with seed 0, so the runs of an algorithm emit the same tokens and differ only in timing.

Timed runs are isolated and paired. Each is its own queue job on a whole H200, whose only other context is an idle placement anchor that launches no kernels; no two of the 60 SVIP and SVIP LK (ours) runs shared a GPU. Run k of SVIP and of SVIP LK (ours), for the same model pair and temperature, started within 15 minutes of each other (median 3) on two GPUs of one machine, and each speedup divides by the autoregressive run k of the same target and temperature.

The systems profiles behind panels G and I of Figure 7 ran on the H200 machine after a driver update (615.71.10, CUDA 13.4). Each job times the algorithm and an autoregressive pass of the same target on the same GPU and inputs, synchronising CUDA at phase boundaries after three untimed warm-ups; timings exclude loading, tokenization and warm-up. Our Spec-Bench re-runs agree with its leaderboard within 0.2 accepted tokens per target pass (Figure 9b).

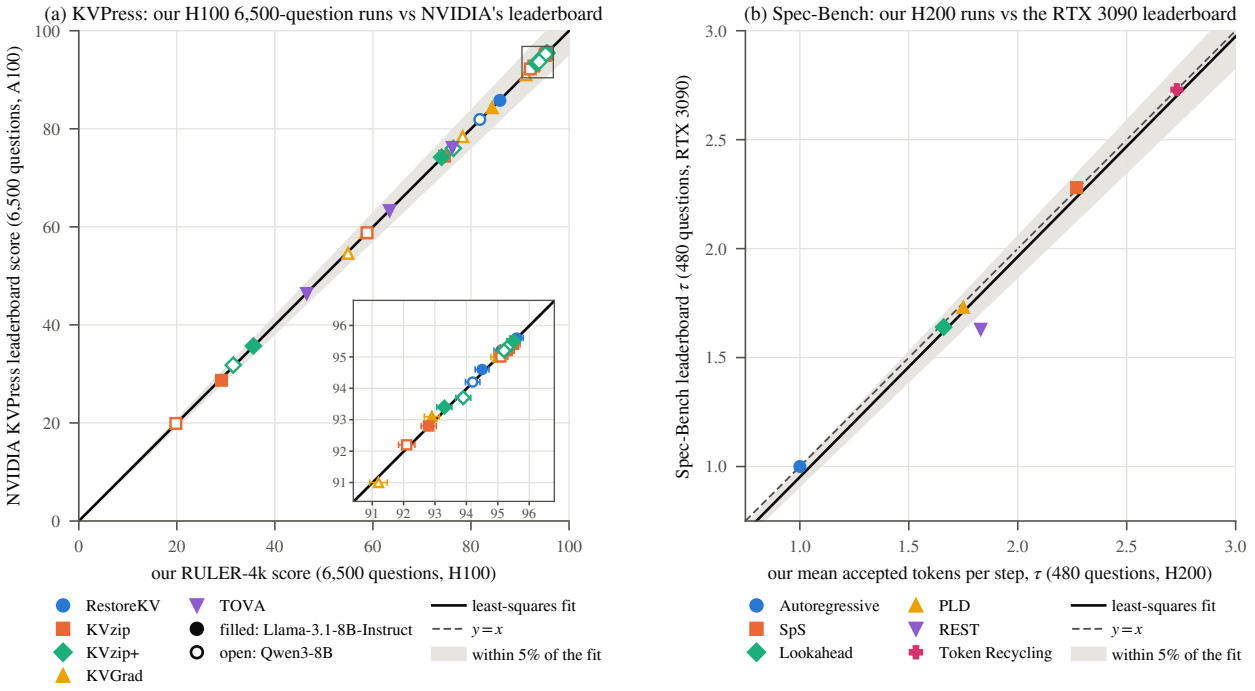

Figure 9. **Our runs reproduce the published leaderboards.** Each point is one published algorithm, in (a) at one model and compression ratio (2×–32×). (a) Horizontal bars: ±1 bootstrap standard deviation of our score over questions (the leaderboard publishes no per-question data); inset: the boxed region, enlarged. (b) Vicuna-7B-v1.3 target, greedy decoding ($T = 0$); each published value is a mean of three runs.

A.2 Reproducing our harness

The harness has three steps, each on its own page with a diagram: preparing the shared problem formulation (Appendix A.2.1) and formalizing a published algorithm (Appendix A.2.2), which run once per field as the offline stage of Section 2.2, and

making variants (Appendix A.2.3), the online stage. In each step an agent follows a prompt of Appendix D: Prompt 1 writes an algorithm's proof document from its code, Prompt 2 transcribes it into Lean, and Prompt 3 derives a variant and its proof. The harness also needs the benchmark, a Lean 4 project with Mathlib, an agent that reads and writes files and runs scripts, and GPUs.

A.2.1 Preparing the shared problem formulation

Start from the field's benchmark (Figure 10a). Take the algorithms it implements and the papers that cite them, and let a person keep those that are relevant and can run on the benchmark. Rewrite each as a self-contained `code.py` that reproduces the benchmark's reported scores or, for a ported algorithm, some of the results of its own paper. Ask an LLM harness for one problem that every algorithm could be proved to solve (Figure 10b), and formalize a random sample of algorithms on it (Appendix A.2.2). Keep the problem and formalize every algorithm on it if a Lean file builds for each of them; otherwise, ask for a new formulation.

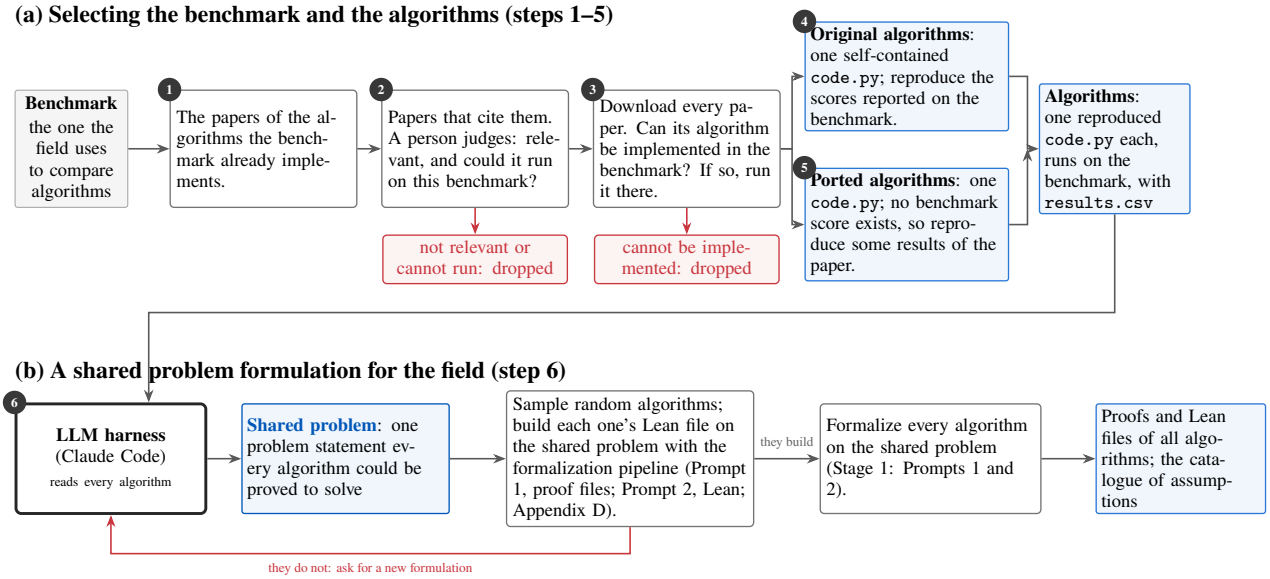

Figure 10. **Stage 1 selects a field's reproducible algorithms and finds one problem that all of them can be proved to solve.** Black discs number the steps, and boxes with a blue border are outputs. Red marks the algorithms dropped at a step and the request for a new formulation when the Lean file of a sampled algorithm does not build.

A.2.2 *Formalizing a published algorithm*

Two agents formalize each algorithm in turn (Figure 11). The first, with Prompt 1, writes the proof document from the algorithm's code, not its paper: every definition transcribes a line of the code, up to a main theorem that the code meets the shared problem's contract. The second, with Prompt 2, reads `proof.tex` and `code.py` and writes `axioms.lean`, with the assumptions and the main theorem's statement, and `proof.lean`, which adds the proofs. Last, we group the assumptions of all algorithms by the design question they answer into one shared pool.

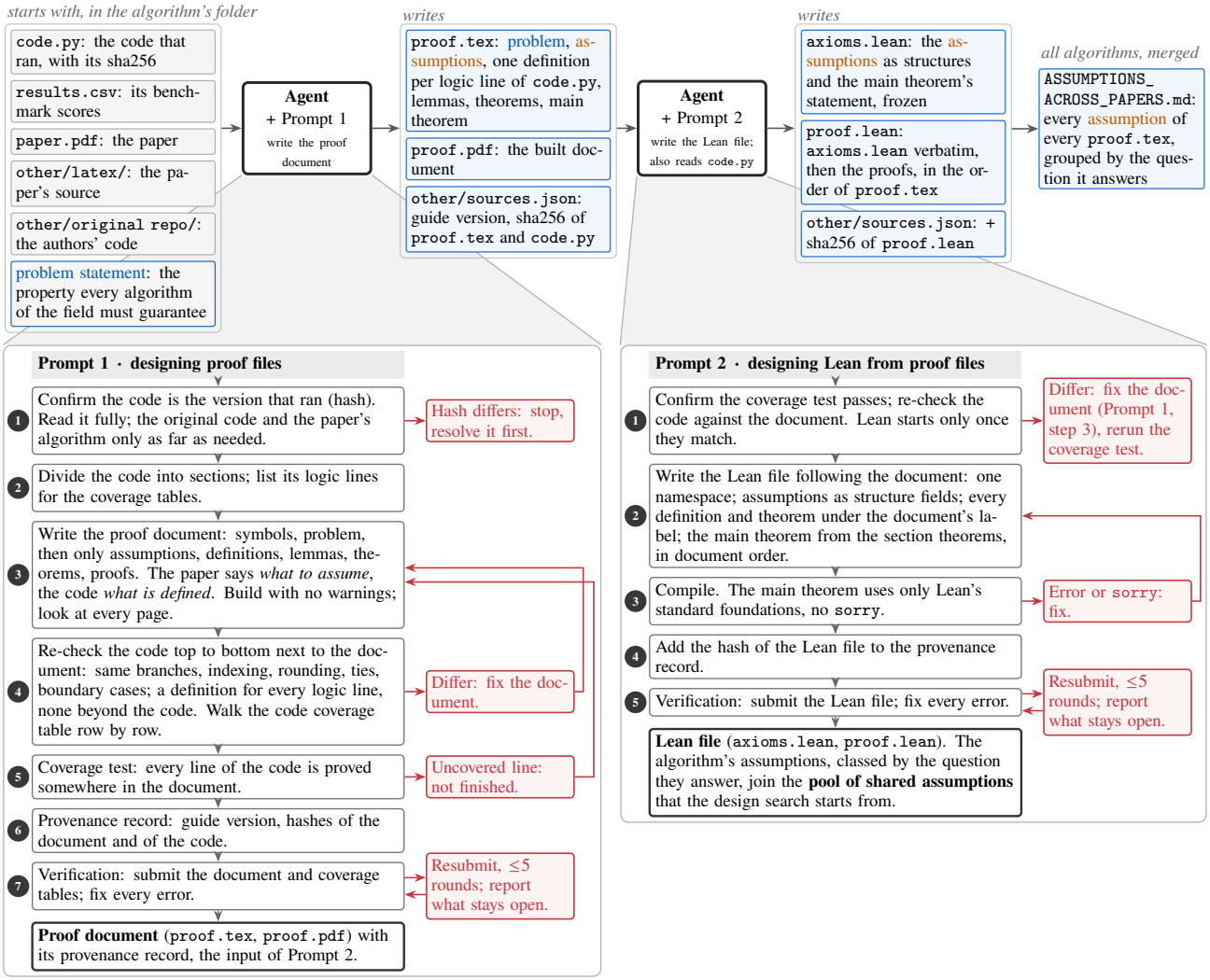

Figure 11. **Formalizing a published algorithm with two agents.** Top: the files each agent reads and writes for one algorithm, and the pool merged over all algorithms. Bottom: each agent box expanded into the workflow of its prompt (Appendices D.1 and D.2), with steps numbered as in the prompt. Grey: files a step starts from; blue: files it writes; red: checks, with arrows to the step a failure returns to.

A.2.3 Making variants

One agent makes each variant in a copy of the parent's folder (Figure 12). Following Prompt 3, it reads the parent's `code.py`, `proof.tex` and scores and the shared pool, chooses one move on one assumption, often borrowed from the pool, and writes the new assumption and its hypothesis before the smallest code change. Before any GPU time, the variant must be no known algorithm, reproduce the parent at one parameter value and differ from it on some input. Its proof edits the parent's only within the impact set of the changed assumption (changes in red, borrowed text in yellow). We benchmark it in the parent's setting, compare it with the literature and verify it; its scores guide the choice of the next parent and assumption.

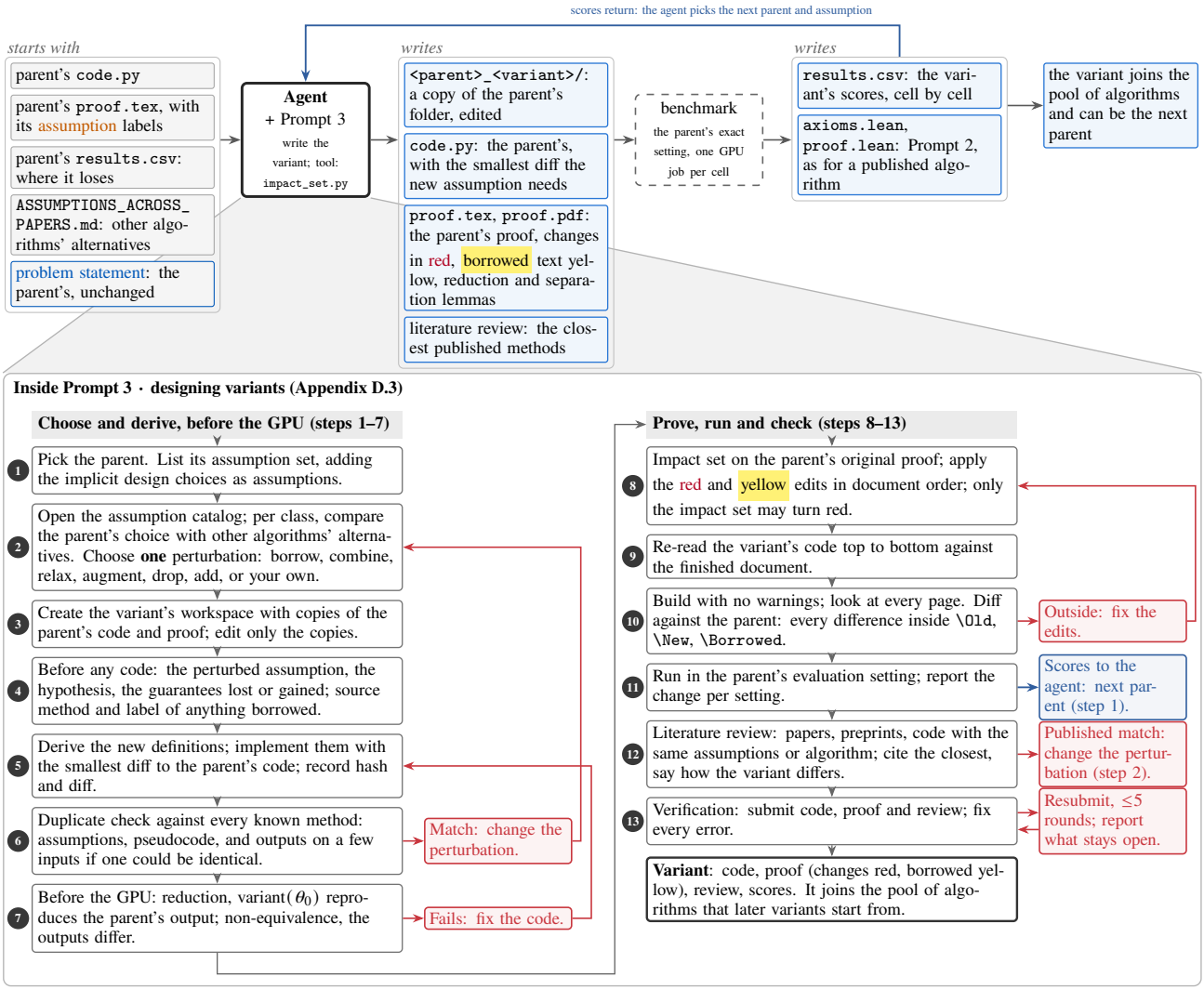

Figure 12. **Making one variant.** Top: the files the agent with Prompt 3 (Appendix D.3) reads and writes. Bottom: the agent box opened into the workflow of Prompt 3, numbered as in the prompt. Grey: files a step starts from; blue: files it writes and the scores returning to the agent; red: checks and the step a failure returns to, as in Figure 11.

B RESULTS IN NUMBERS

This appendix gives the numbers behind the figures: one subsection per field (KV-cache eviction, speculative decoding and LLM weight pruning), one for the harness comparison and one for the best variant of every parent; uncertainties as in Appendix G.

B.1 KV-cache eviction

B.1.1 Main benchmark

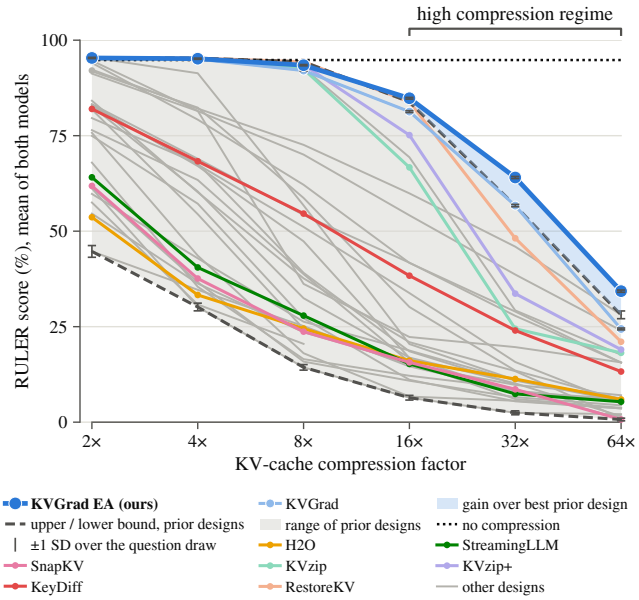

Figure 13. **KVGrad EA (ours) lies above every prior design at 16×–64× compression, where it gains 6.9 points on average over its parent KVGrad (3.7 points over 2×–64×).** RULER-4K, mean of Llama-3.1-8B-Instruct and Qwen3-8B. The 33 query-agnostic prior designs (grey unless the legend names them) are each drawn at the largest number of questions both models finished (300 or 6,500); KVGrad EA (ours) and KVGrad use 6,500. Table 5 lists the named prior algorithms per model.

We ran 34 published KV-cache eviction algorithms on all twelve RULER-4K cells; RestoreKV GM (ours) (75.7) has a higher mean than every one of them, the best being KVGrad (74.1). Table 5 lists the published algorithms Figure 13 names, with RestoreKV GM (ours), which has the best mean among them. It holds the best score in four of the six 16×–64× cells; KVGrad keeps 64× on Llama (26.0 against 25.2) and 32× on Qwen (54.9 against 54.0). At 2×–8× the published algorithms are as good. Three algorithms drawn only in grey would change a mark: CaM (TOVA) (38.4) and NestedKV (33.8) at 64× on Llama, and DefensiveKV (Feng et al., 2026) (96.0) at 2× on Llama, all at 300 questions.

Table 5. **RULER-4K scores of the published algorithms named in Figure 13 and of RestoreKV GM (ours), per model and compression ratio.** Each algorithm at the largest number of questions both models finished (Questions; 300 is nested in the 6,500); Mean covers the 12 cells. ±: one bootstrap SD over the question draw. Bold: best in the column; underlined: second best, at the printed precision.

		Llama-3.1-8B-Instruct						Qwen3-8B						Mean
	Questions	2×	4×	8×	16×	32×	64×	2×	4×	8×	16×	32×	64×	
H2O	300	54.6 ±2.1	35.6 ±1.6	25.5 ±1.4	14.7 ±1.3	10.4 ±1.4	4.2 ±1.0	52.7 ±2.0	31.0 ±1.7	23.5 ±1.4	17.5 ±1.3	12.2 ±1.2	7.7 ±1.1	24.13 ±0.43
SnapKV	300	70.0 ±1.9	42.3 ±2.0	25.7 ±1.8	15.9 ±1.6	8.9 ±1.3	0.0 ±0.0	53.7 ±2.0	32.9 ±1.9	21.7 ±1.6	15.4 ±1.5	8.3 ±1.2	1.4 ±0.7	24.68 ±0.45
StreamingLLM	300	63.4 ±2.4	39.4 ±2.2	28.0 ±1.9	16.4 ±1.6	8.4 ±1.4	5.5 ±1.2	64.8 ±2.4	41.6 ±2.2	27.8 ±2.0	14.1 ±1.6	6.4 ±1.3	5.2 ±1.1	26.75 ±0.52
KeyDiff	300	86.7 ±1.4	73.5 ±1.4	62.8 ±1.1	52.4 ±1.2	33.3 ±1.4	17.8 ±1.5	77.3 ±1.6	63.1 ±1.4	46.4 ±1.8	24.3 ±1.5	14.7 ±1.1	8.7 ±0.4	46.75 ±0.40
KVzip	6,500	95.5 ±0.2	95.3 ±0.2	92.8 ±0.2	74.6 ±0.4	29.1 ±0.4	17.7 ±0.2	95.2 ±0.2	95.1 ±0.2	92.1 ±0.3	58.8 ±0.5	19.8 ±0.2	18.6 ±0.2	65.38 ±0.08
KVzip+	6,500	95.5 ±0.2	95.2 ±0.2	93.3 ±0.2	74.0 ±0.3	35.6 ±0.4	18.2 ±0.2	**95.3** ±0.2	95.2 ±0.2	93.9 ±0.2	76.4 ±0.4	31.5 ±0.4	19.8 ±0.2	68.66 ±0.08
RestoreKV	6,500	**95.6** ±0.2	95.3 ±0.2	**94.5** ±0.2	85.9 ±0.3	52.8 ±0.5	19.4 ±0.2	95.1 ±0.2	**95.3** ±0.2	**94.2** ±0.2	81.8 ±0.3	43.5 ±0.4	22.7 ±0.3	73.01 ±0.08
KVGrad	6,500	95.5 ±0.2	95.1 ±0.2	92.9 ±0.2	84.3 ±0.3	58.5 ±0.4	**26.0** ±0.3	**95.3** ±0.2	95.0 ±0.2	91.2 ±0.3	78.3 ±0.4	**54.9** ±0.4	22.8 ±0.3	74.15 ±0.09
RestoreKV GM (ours)	6,500	95.5 ±0.2	**95.4** ±0.2	94.4 ±0.2	**86.6** ±0.2	**64.8** ±0.4	25.2 ±0.3	95.2 ±0.2	**95.3** ±0.2	94.1 ±0.2	**84.7** ±0.3	54.0 ±0.4	**23.7** ±0.3	**75.74** ±0.08

B.1.2 Compression ratios and unseen settings

Table 6 gives the scores behind Figure 7B (Llama) and the same cells for Qwen, with KVzip and KVzip+ for reference. RestoreKV GM (ours) ties RestoreKV at 2×–8× and gains up to 12.0 points at 32×; its mean is 2.73 points higher. Errors: resampling the questions (Appendix G.1).

Table 6. **Full RULER-4K scores (6,500 questions) of KVzip, KVzip+, RestoreKV and RestoreKV GM (ours), per model and compression ratio.** Small figures: RestoreKV GM minus RestoreKV, green where a two-sided z-test gives $p < 0.05$, grey otherwise (Appendix G). Table 5 gives the error bars.

KV cache eviction	Llama-3.1-8B-Instruct						Qwen3-8B						Mean
	2×	4×	8×	16×	32×	64×	2×	4×	8×	16×	32×	64×	
KVzip	95.5	95.3	92.8	74.6	29.1	17.7	95.2	95.1	92.1	58.8	19.8	18.6	65.38
KVzip+	95.5	95.2	93.3	74.0	35.6	18.2	95.3	95.2	93.9	76.4	31.5	19.8	68.66
RestoreKV	95.6	95.3	94.5	85.9	52.8	19.4	95.1	95.3	94.2	81.8	43.5	22.7	73.01
RestoreKV GM (ours)	95.5 -0.1	95.4 +0.1	94.4 -0.1	86.6 +0.7	64.8 +12.0	25.2 +5.8	95.2 +0.1	95.3 0.0	94.1 -0.1	84.7 +2.9	54.0 +10.5	23.7 +1.0	**75.74** +2.73

In every RULER setting (Table 7) the gain is mostly at 32× and 64× (+5.8 to +12.0 points, except +1.0 for Qwen at 64×), with 2×–8× within 0.3 points; on LongBench it is spread more evenly and is largest at 16× (+3.3). Errors: resampling the questions (Appendix G.1).

Table 7. **Scores of RestoreKV and RestoreKV GM (ours) at 8K and 16K contexts, on Qwen3-8B and on LongBench, per compression ratio.** Mean is the value in the top of Table 3. ±: one bootstrap SD over the question draw. Bold: the higher of the two.

Setting	Questions	Design	2×	4×	8×	16×	32×	64×	Mean
RULER 4K Llama	6,500	RestoreKV	**95.6** ±0.2	95.3 ±0.2	**94.5** ±0.2	85.9 ±0.3	52.8 ±0.5	19.4 ±0.2	73.9 ±0.11
		RestoreKV GM (ours)	95.5 ±0.2	**95.4** ±0.2	94.4 ±0.2	**86.6** ±0.2	**64.8** ±0.4	**25.2** ±0.3	**77.0** ±0.12
RULER 8K Llama	6,500	RestoreKV	**94.6** ±0.2	**93.9** ±0.2	**92.9** ±0.2	84.1 ±0.3	61.1 ±0.4	22.4 ±0.3	74.8 ±0.12
		RestoreKV GM (ours)	94.5 ±0.2	**93.9** ±0.2	92.8 ±0.2	**84.6** ±0.3	**68.1** ±0.4	**33.5** ±0.4	**77.9** ±0.12
RULER 16K Llama	6,500	RestoreKV	**93.2** ±0.2	**92.9** ±0.2	**90.0** ±0.3	82.1 ±0.3	59.2 ±0.4	22.1 ±0.3	73.3 ±0.12
		RestoreKV GM (ours)	**93.2** ±0.2	**92.9** ±0.2	89.7 ±0.3	**83.5** ±0.3	**66.4** ±0.4	**33.5** ±0.4	**76.5** ±0.13
RULER 4K Qwen	6,500	RestoreKV	95.1 ±0.2	**95.3** ±0.2	**94.2** ±0.2	81.8 ±0.3	43.5 ±0.4	22.7 ±0.3	72.1 ±0.12
		RestoreKV GM (ours)	**95.2** ±0.2	**95.3** ±0.2	94.1 ±0.2	**84.7** ±0.3	**54.0** ±0.4	**23.7** ±0.3	**74.5** ±0.12
LongBench Llama	3,750	RestoreKV	47.3 ±0.5	47.7 ±0.5	45.8 ±0.6	37.3 ±0.6	26.8 ±0.4	21.8 ±0.3	37.8 ±0.20
		RestoreKV GM (ours)	**47.9** ±0.5	**47.9** ±0.5	**46.8** ±0.5	**40.6** ±0.6	**29.0** ±0.5	**23.3** ±0.4	**39.2** ±0.20

B.1.3 Systems cost

Table 8 gives the time to first token of no eviction, four published algorithms and RestoreKV GM (ours) across context lengths; Figure 7C plots these times, and Section 4.1 tests it. RestoreKV GM stays within the run-to-run spread of RestoreKV at every context. KVGrad is the slowest by far: its compression takes 3 to 14 min at 32K and 64K.

Table 8. **Time to first token of no eviction, KVGrad, RestoreKV, KVzip, KVzip+ and RestoreKV GM (ours) at 4K to 64K tokens.** Seconds, mean ± SD over seven timing passes of two fixed RULER prompts, Llama-3.1-8B-Instruct on one H100, no eviction at full cache and every other method at 16× compression; the first row is the KV-cache size reduction of the compressing methods. Ours in bold.

Context	4K	8K	16K	32K	64K
KV-cache size reduction (GB)	0.46	0.97	1.95	3.99	8.03
No eviction (no compression)	0.23 ±0.11	0.40 ±0.13	0.75 ±0.04	3.48 ±2.06	5.94 ±0.90
KVGrad	3.93 ±1.29	11.7 ±4.36	35.1 ±3.15	175.4 ±0.89	814.3 ±1.97
RestoreKV	1.81 ±0.69	3.51 ±1.62	5.16 ±2.04	12.2 ±1.27	33.0 ±0.38
KVzip	1.87 ±0.79	3.36 ±1.20	5.13 ±1.36	11.4 ±1.10	32.7 ±0.31
KVzip+	2.54 ±1.33	3.73 ±1.27	6.49 ±1.88	13.3 ±0.74	36.6 ±0.51
RestoreKV GM (ours)	**2.52** ±1.50	**3.52** ±1.33	**4.94** ±0.80	**10.9** ±0.12	**33.0** ±0.51

B.2 Speculative decoding

The bars of Figure 7G are the published decoders Hydra (Ankner et al., 2024), EAGLE-2 (Li et al., 2024b), Medusa (Cai et al., 2024), MCSD (Yang et al., 2024), acceptance feedback (Gante, 2023), SVIP (Zhang et al., 2025), confidence stopping (Mamou et al., 2024b;a), AdaEDL (Agrawal et al., 2024), speculative sampling with fixed $K = 5$ (Leviathan et al., 2023; Chen et al., 2023) and PLD (Saxena, 2023). SVIP LK (ours) adds a prompt-lookup fallback to SVIP (Algorithm 3). Figure 7G uses the systems-profile runs (Appendix A.1).

B.3 LLM weight pruning

Pruner-Zero LG (ours) is the best pruner in every perplexity column, and its lead grows with sparsity. Tables 9 and 10 give the cells behind panels D and E of Figure 7 and the bottom of Table 3. The margin over the second best grows from below 0.01 at 10 % to 0.10 at 40 % and 0.11 at 50 %. On the zero-shot tasks (Table 11) no pruner is best on every task; Pruner-Zero LG (ours) gains on its parent on three of the seven tasks and on the mean ($+0.3$). Each cell is one run with a fixed calibration seed (Section 3); the LLaMA-30B cells of ALPS, GBLM-Pruner, Pruner-Zero and Pruner-Zero LG (ours) ran on the first H100 machine and the other LLaMA-30B cells on a second one (Appendix A.1).

B.3.1 Model sizes

Table 9. **WikiText-2 test perplexity of eleven published pruners and Pruner-Zero LG (ours) on LLaMA 7B, 13B and 30B and LLaMA-2 7B at 50 % unstructured sparsity.** The values behind the bottom of Table 3 and Figure 7D; lower is better. Rows: the two baselines, the eleven published algorithms and **ours**, Pruner-Zero LG (ours), in bold. **Bold** = best, underlined = second best in the column, Dense excluded. One seed-0 run per cell, 128 C4 calibration windows of 2,048 tokens, fp16, H100.

Method	LLaMA-7B	LLaMA-13B	LLaMA-30B	LLaMA-2-7B
Dense	5.68	5.09	4.10	5.47
Magnitude	17.26	20.14	7.55	16.03
SparseGPT	7.22	6.21	5.32	7.02
E-Sparse	7.26	6.15	5.24	6.91
GBLM-Pruner	7.15	6.11	5.20	6.86
Wanda	7.26	6.15	5.25	6.92
RI	7.49	6.57	5.27	7.39
RIA	7.12	6.08	5.08	6.81
OWL + Wanda	7.22	6.08	5.25	6.86
DSnoT + Wanda	7.15	6.09	5.21	6.85
Pruner-Zero	6.89	5.93	5.02	6.71
ALPS	7.14	6.14	5.30	6.87
Z-Pruner	7.11	6.04	5.09	6.77
Pruner-Zero LG	**6.78**	**5.83**	**4.93**	**6.67**

B.3.2 Sparsity ratios

Table 10. **WikiText-2 test perplexity of eleven published pruners and Pruner-Zero LG (ours) on LLaMA-7B at 10, 20, 30 and 40 % unstructured sparsity.** The values of Figure 7E; lower is better; the 50 % column is in Table 9. Same rows, runs and marking as Table 9.

Method	10 %	20 %	30 %	40 %
Dense	5.68	5.68	5.68	5.68
Magnitude	5.81	6.02	6.67	8.59
SparseGPT	5.70	5.80	5.97	6.32
E-Sparse	5.70	5.82	6.00	6.39
GBLM-Pruner	5.69	5.76	5.90	6.27
Wanda	5.70	5.82	6.00	6.39
RI	5.69	5.74	5.89	6.28
RIA	5.69	5.75	5.86	6.21
OWL + Wanda	5.71	5.81	6.02	6.39
DSnoT + Wanda	5.69	5.76	5.89	6.27
Pruner-Zero	5.68	5.73	5.85	6.18
ALPS	5.71	5.84	6.01	6.37
Z-Pruner	5.69	5.75	5.87	6.22
Pruner-Zero LG	**5.68**	**5.72**	**5.82**	**6.08**

B.3.3 Zero-shot accuracy

Table 11. **Zero-shot accuracy on seven tasks of eleven published pruners and Pruner-Zero LG (ours) on LLaMA-7B at 50 % unstructured sparsity.** lm-eval accuracy (%, higher is better) on BoolQ, RTE, HellaSwag, WinoGrande, ARC-easy, ARC-challenge and OpenBookQA, and their mean; same rows, runs and marking as Table 9.

Method	BoolQ	RTE	HellaSwag	WinoGr.	ARC-e	ARC-c	OBQA	Mean
Dense	76.7	67.1	57.1	70.2	75.5	42.5	33.8	60.4
Magnitude	56.4	58.8	45.0	58.6	60.5	32.2	25.6	48.2
SparseGPT	73.5	59.2	51.7	67.6	71.2	37.7	**31.0**	56.0
E-Sparse	71.5	57.0	52.0	67.6	70.5	38.0	30.6	55.3
GBLM-Pruner	**74.4**	59.6	52.2	67.2	71.4	38.1	**31.0**	56.3
Wanda	71.7	57.0	52.0	67.6	70.8	37.6	30.0	55.2
RI	68.3	56.3	**52.5**	66.6	68.7	36.6	28.4	53.9
RIA	73.1	60.6	51.9	68.4	71.1	38.2	29.2	56.1
OWL + Wanda	72.6	57.8	52.0	69.5	71.2	36.9	29.2	55.6
DSnoT + Wanda	73.6	59.2	52.1	66.8	**72.1**	37.9	30.8	56.1
Pruner-Zero	71.8	60.6	51.0	67.1	70.7	36.3	27.6	55.0
ALPS	72.2	49.1	52.4	**69.5**	70.4	38.0	**31.0**	54.7
Z-Pruner	72.2	**63.5**	52.0	68.3	71.4	**38.9**	28.4	**56.4**
Pruner-Zero LG	71.4	61.7	50.7	66.9	70.8	35.8	29.8	55.3

B.4 Harness comparison

OpenEvolve has the highest mean from the fifth algorithm on (Table 12). Ours ends 0.4 points below it, inside its range, with the narrowest range between repeats. Errors: repeating the search (Appendix G.3).

Table 12. **Running best RULER-4k score after 1 to 10 algorithms of Fable from scratch, AIDE, OpenEvolve and our harness (the numbers of Figure 8).** Score: the mean of the 12 cells, Llama-3.1-8B-Instruct and Qwen3-8B at 2×–64× (200 questions); each cell is the mean over three repeats per search, with the minimum and maximum across the repeats beneath it in small grey print. OpenEvolve, AIDE and ours start from RestoreKV, whose own score is excluded. Per column, bold: the highest mean; underlined: the second highest, ranked before rounding. ⋆: found by our harness.

	Designs evaluated									
	1	2	3	4	5	6	7	8	9	10
Fable, from scratch	49.3 44.0–58.8	54.1 44.0–59.2	62.5 58.6–65.8	63.1 59.0–67.2	64.7 60.3–67.2	66.4 63.4–68.1	68.2 67.6–68.8	69.4 67.6–71.9	71.4 68.8–73.0	71.4 68.8–73.0
AIDE	**72.8** 70.4–77.7	<u>75.4</u> 70.4–78.2	<u>76.3</u> 72.7–78.2	**76.6** 73.0–78.8	<u>76.9</u> 73.2–79.2	<u>77.1</u> 73.2–79.9	<u>78.2</u> 73.6–80.8	<u>78.2</u> 73.6–80.8	78.2 73.6–80.8	78.5 73.8–80.9
OpenEvolve	<u>70.7</u> 66.5–72.7	**76.1** 72.7–77.9	**76.6** 74.1–77.9	<u>76.6</u> 74.1–77.9	**77.3** 76.2–77.9	**78.7** 77.7–80.4	**78.8** 77.9–80.4	**79.5** 78.5–81.3	**79.7** 78.8–81.3	**79.7** 78.8–81.3
Ours⋆	69.6 66.8–71.1	70.7 70.1–71.1	71.0 71.0–71.1	72.4 71.2–74.8	72.9 71.2–74.8	75.1 75.0–75.2	77.2 77.1–77.2	77.5 77.2–77.8	<u>79.3</u> 78.8–80.1	<u>79.3</u> 78.8–80.1

B.5 Best variant of every parent

Figure 6 shows the seven parents with the largest gain per field; Table 13 lists all 26 parents. Every parent has a variant that scores higher on its field's search benchmark, although the smallest gains lie within the benchmark noise.

Table 13. **The best variant of each of the 26 parent algorithms on its field's search benchmark.** Variants: how many the harness derived from the parent; Better: how many score above it. Gain: best variant minus parent, in RULER-4K points (200 questions, mean of both models and six compression ratios), WikiText-2 perplexity of LLaMA-7B at 50 % sparsity (sign flipped, so positive is better) and speedup over autoregressive decoding (Vicuna-7B, mean of five temperatures); Gain (%): relative to the parent.

Parent	Best variant	Variants	Better	Gain	Gain (%)
KV-cache eviction (gain in RULER points)					
PyramidKV	PyramidKV lastquery	6	4	+12.43	+54.8
KNorm	KNorm value norm	6	5	+8.41	+44.0
TOVA	TOVA pooling	6	2	+5.16	+15.5
SnapKV	SnapKV head mean	6	6	+2.99	+12.3
KVzip+	KVzip+ residual norm	40	22	+4.45	+6.5
KVGrad	KVGrad exact attention	40	13	+3.57	+4.8
RestoreKV	RestoreKV GM (ours)	36	16	+3.52	+4.8
EchoPress	EchoPress group mean	6	3	+2.91	+4.4
KVzip	KVzip group mean	6	3	+2.27	+3.5
Expected Attention	Expected Attention group max	6	2	+0.46	+1.0
CaM+TOVA	CaM+TOVA head max	6	1	+0.61	+1.0
Weight pruning (gain in perplexity, lower perplexity counted as positive)					
RI	RI act inside	10	5	+0.293	+3.9
Pruner-Zero	Pruner-Zero LG (ours)	10	2	+0.103	+1.5
RIA	RIA owl alloc	10	5	+0.093	+1.3
Wanda	Wanda colnorm	10	3	+0.072	+1.0
ALPS	ALPS diag shrink	10	5	+0.066	+0.9
DSnoT (Wanda)	DSnoT (Wanda) down proj	10	2	+0.039	+0.6
Z-Pruner	Z-Pruner alpha half	10	5	+0.019	+0.3
Speculative decoding (gain in speedup)					
SVIP	SVIP LK (ours)	7	3	+0.216	+14.5
Tree Monte Carlo	Tree Monte Carlo mixture target	7	3	+0.162	+12.2
Acceptance-feedback γ	Acceptance-feedback γ cumconf cut	7	3	+0.082	+5.5
Confidence stop	Confidence stop discounted chain	7	6	+0.078	+5.1
AdaEDL	AdaEDL maxconf	7	3	+0.074	+5.0
SpS	SpS prompt lookup	7	1	+0.044	+3.2
SpS fixed k	SpS fixed k lookup tail	7	1	+0.046	+3.1
Block verification	Block verification margin stop	7	1	+0.014	+1.0

C ALGORITHM SKETCHES

Pseudocode of the three variants the paper discusses in detail, one per field: RestoreKV GM (ours) (KV-cache eviction), Pruner-Zero LG (ours) (weight pruning) and SVIP LK (ours) (speculative decoding).

C.1 KV-cache eviction

C.1.1 RestoreKV GM (ours)

Algorithm 1 RestoreKV GM (ours).

Require: model with L layers, H KV heads per layer, G query heads per KV head, head dimension d; context tokens $x_1, \dots, x_n$, of which the first p are the chat-template prefix; eviction fraction $r \in [0, 1)$; $n_s = 4$ sink positions; replay chunk size $c = 2048$; $n_E = 8$ trained restore embeddings E and LoRA restore adapter Δ

Require: group aggregation $\in \{\mathrm{mean}, \max\}$, set to mean ($\max$ recovers RestoreKV)

Ensure: for $r > 0$, the KV cache with n_E restore pairs added per layer and KV head and the $\lfloor \min(1, r + n_E/n)\, LHn \rfloor$ lowest-scored context pairs masked out

Prefill: run the model on $x_1, \dots, x_n$ and keep keys $k^l_{h,i}$ and values $v^l_{h,i}$ for every layer l, KV head h and position $i \le n$
if $r = 0$, **return** the cache unchanged ▷ no replay, no restore pass
$S^l_{h,i} \leftarrow 0$ for all l, h, i ▷ prefix positions are never scored; those past the sinks keep 0
split $x_{p+1}, \dots, x_n$ into consecutive chunks $C_1, \dots, C_m$ of c tokens each (the last may be shorter)
for $j = 1, \dots, m$ **do**
  $P \leftarrow$ the tokens of "\n\nRepeat the previous context exactly." if $j = 1$; else the tokens of "\n\nRepeat the part of the previous context exactly, starting with" followed by the last 8 tokens of C_{j-1}
  replay $R \leftarrow P$, the chat-template text that follows a user message (end of the user turn, assistant header), C_j; its q tokens take positions $n + 1, \dots, n + q$
  run the model on R on top of the cache; after the attention sublayer of every layer l has run (its output unchanged):
    $Q_{hg,t} \leftarrow$ query of replay token t for query head (h, g), rotary-encoded at position $n + t$
    $\mathcal{K} \leftarrow$ the first $\min(n_s, |\text{positions before } C_j|)$ cached positions, the positions of C_j and the q replay positions, whose keys the replay tokens have just written
    $a_{hg}(t, i) \leftarrow \mathrm{softmax}_{i \in \mathcal{K}}\big(Q_{hg,t} \cdot k^l_{h,i}/\sqrt{d}\big)$, replay positions masked causally
    keep $a_{hg}(t, i)$ only for the positions i of C_j
    $m_{hg}(i) \leftarrow \max_t\ a_{hg}(t, i)$ for i in C_j (per query head)
    $S^l_{h,i} \leftarrow \frac{1}{G} \sum_{g=1}^{G} m_{hg}(i)$ for i in C_j (mean over the G query heads of the group)
    drop the replay tokens' keys and values from the cache (it holds the n context pairs again)
end for
Restore pass: switch on Δ, run the model on E at positions $n + 1, \dots, n + n_E$ on top of the full cache (no scoring), append the n_E new pairs per layer and KV head to the cache, switch Δ off
$r' \leftarrow \min(1, r + n_E/n)$ ▷ the restore pairs use part of the budget
$S^l_{h,i} \leftarrow \max S + 1$ for $i \le n_s$ ▷ rank sinks above other context pairs
$N \leftarrow \lfloor r' \cdot LHn \rfloor$; $\mathcal{P} \leftarrow$ the N triples (l, h, i) with the lowest $S^l_{h,i}$, ranked jointly over all layers, heads and positions
for every layer l **do**
  $N_l \leftarrow$ the number of triples of $\mathcal{P}$ in layer l
  mask the N_l pairs (h, i) of layer l with the lowest $S^l_{h,i}$: later attention ignores them
end for
return the cache

C.2 LLM weight pruning

C.2.1 Pruner-Zero LG (ours)

Algorithm 2 Pruner-Zero LG (ours).

Require: dense model in fp16 with L decoder layers; the weight matrices W_λ^l of every linear λ of layer l (query, key, value, output, gate, up and down projections), each with R output rows and C input columns; sparsity $s \in [0, 1)$ (unstructured) or an $N{:}M$ pattern; $n_g = 128$ gradient windows and $n_c = 128$ calibration windows of $T = 2048$ tokens, seed 0; gradient scale $\alpha = 100$

Require: comparison group = matrix: a weight matrix loses its $\lfloor sRC \rfloor$ lowest-scored weights as a whole (`layer` in the code; each output row losing its $\lfloor sC \rfloor$ lowest, `row`, recovers Pruner-Zero)

Ensure: the model with, in every decoder linear, $\lfloor sRC \rfloor$ weights set to 0 (or N of every M consecutive inputs of every row); embeddings, norms and the output head unchanged

Gradient pass: draw n_g random windows of T tokens from the WikiText-2 training text; for every window w, take the gradient $g_{\lambda,w}^l$ of the dense model's language-model loss on w with respect to every W_λ^l

$G_\lambda^l \leftarrow \sqrt{\sum_w (\alpha\, g_{\lambda,w}^l)^2}$ elementwise, rounded to fp16 ▷ one $R \times C$ gradient norm per matrix, computed once per model

Calibration inputs: draw n_c random windows of T tokens from the C4 training text and record the inputs X_1 of decoder layer 1 for all of them

for $l = 1, \ldots, L$ **do**

  for every weight matrix $W = W_\lambda^l$ of layer l, in module order **do**

    $A_{ij} \leftarrow |W_{ij}|^2$ ($|W|$ in fp16, squared in fp32) ▷ the magnitude term of the evolved metric

    $G \leftarrow G_\lambda^l$ in fp32; $\tilde{G}_{ij} \leftarrow (G_{ij} - \min G)/(\max G - \min G)$, the minimum and maximum over the whole matrix

    $S_{ij} \leftarrow A_{ij}\, \tilde{G}_{ij}$ ▷ Pruner-Zero's evolved score $|(|W| \times |W|)| \times \mathrm{mms}(G)$; no activations

    if unstructured **then**

      $k \leftarrow \lfloor sRC \rfloor$; $\mathcal{P} \leftarrow$ the k positions (i, j) with the lowest S_{ij}, ranked jointly over the whole matrix, ties to the earlier position in row-major order

    else

      $\mathcal{P} \leftarrow$ in every row i and every block of M consecutive columns, the N positions with the lowest S_{ij}

    end if

    $W_{ij} \leftarrow 0$ for $(i, j) \in \mathcal{P}$ ▷ the remaining weights are not updated

  end for

  $X_{l+1} \leftarrow$ the outputs of the pruned layer l on X_l ▷ layers are pruned in order; the score reads no activations, so the pass only feeds the next layer

end for

return the pruned model

C.3 Speculative decoding

C.3.1 SVIP LK (ours)

Algorithm 3 SVIP LK (ours).

Require: target model $\mathcal{M}_\theta$ (logits $z(u)$ for the token after a sequence u) and drafter $\mathcal{M}_q$ (logits $z_q(u)$), each with a KV cache; prompt $x_1, \ldots, x_n$; cap $N = 1024$ on new tokens; EOS token e; draft cap $D_{\max} = 40$ per step; entropy threshold $h = 0.3$ and constant $c_H = 0.15$ of the stop test; setting σ: greedy ($T = 0$, the Spec-Bench protocol) or sample with temperature τ and top-k $\kappa = 50$ ($w_{\tau,\kappa}(v)$: softmax of v/τ restricted to its κ largest entries)

Require: lookup fallback: at most $\Lambda = 10$ lookup tokens per step (lookup_tokens; $\Lambda = 0$ recovers SVIP), largest n-gram size $\nu = 3$ of the lookup (lookup_ngram)

Ensure: the sequence s (prompt followed by the emitted tokens), the number of emitted tokens $|s| - n$, the number of steps J (target forward passes after the prefill) and the tokens emitted per step $\ell_1, \ldots, \ell_J$; at $T = 0$ the emitted tokens are the target's greedy continuation of the prompt

Prefill: $s \leftarrow x_{1:n}$; $\mathcal{C}_\theta, \mathcal{C}_q \leftarrow$ caches of $x_{1:n-1}$ by one target and one drafter pass; $c \leftarrow 0$; $j \leftarrow 0$ ▷ c: upstream counter
repeat
  $j \leftarrow j + 1$; $m \leftarrow \min(D_{\max},\ N - c - 1)$ ▷ draft budget of the step
  Draft: $d \leftarrow ()$; $i \leftarrow 0$
  while $i < m$ **do**
    $i \leftarrow i + 1$; $v \leftarrow z_q(s \cdot d_{1:i-1})$ by one drafter pass over the tokens of $s \cdot d_{1:i-1}$ not yet in $\mathcal{C}_q$; $\mathcal{C}_q \leftarrow$ cache of $s \cdot d_{1:i-1}$
    $H \leftarrow -\sum_x \pi(x) \ln \pi(x)$ for $\pi = \mathrm{softmax}(v)$ (nats; raw logits, no τ or top-k) ▷ the drafter's own uncertainty about d_i
    $d_i \leftarrow \arg\max v$ (sample: $q_i \leftarrow w_{\tau,\kappa}(v)$, draw $d_i \sim q_i$); $d \leftarrow d \cdot d_i$
    if $i = m$ **or** $d_i = e$**: break** ▷ count cap, EOS
    if $\sqrt{c_H H} > h$ **then**
      $b \leftarrow \min(\Lambda,\ m - i)$ (the room left under the budget; $b \geq 1$ unless $\Lambda = 0$)
      if $b \geq 1$: **lookup** in $u \leftarrow s \cdot d$, for $\nu' = \nu, \nu - 1, \ldots, 1$ in turn: $a^* \leftarrow$ the earliest a such that $u_{a:a+\nu'-1}$ equals the suffix $u_{|u|-\nu'+1:|u|}$ (an earlier copy of the last ν' tokens), with $a \leq |u| - 2\nu'$ and $a + \nu' + \Lambda - 1 \leq |u|$
        $w \leftarrow u_{a^*+\nu':a^*+\nu'+\Lambda-1}$ at the first ν' with such an a^* (the Λ tokens that followed the copy); $w \leftarrow ()$ if no size has one
        $w \leftarrow$ the first b tokens of w, cut after their first e; $d \leftarrow d \cdot w$ (no drafter pass for these tokens; sample: $q_{i'} \leftarrow \delta_{d_{i'}}$, the point mass, for $i < i' \leq |d|$)
      break
    end if
  end while
  $k \leftarrow |d|$ (i drafter tokens followed by $k - i$ lookup tokens)
  Verify: $(z_0, \ldots, z_k) \leftarrow$ the target logits at positions $|s|, \ldots, |s| + k$ by one target pass over $\mathrm{last}(s) \cdot d$ on top of $\mathcal{C}_\theta$; $\mathcal{C}_\theta \leftarrow$ cache of $s \cdot d$
  Accept: $t_a \leftarrow \arg\max z_a$ for $a = 0, \ldots, k$; $r \leftarrow \max\{a : d_b = t_{b-1} \text{ for all } b \leq a\}$; $y \leftarrow (t_0, \ldots, t_r)$
    (sample, $k \geq 1$: $p_a \leftarrow w_{\tau,\kappa}(z_a)$; $r \leftarrow$ the number of leading coins $\xi_b < p_{b-1}(d_b)/q_b(d_b)$, ξ_b uniform on $[0, 1)$; $y \leftarrow d_{1:r} \cdot c^*$ with $c^* \sim \mathrm{norm}\big((p_r - q_{r+1})_+\big)$ if $r < k$, else $c^* \sim p_k$)
  if $\mathrm{last}(s \cdot d) = e$ **and** $r = k$: $y \leftarrow y_{1:|y|-1}$ ▷ no bonus token after an accepted EOS draft
  $\ell_j \leftarrow |y|$; $s \leftarrow s \cdot y$ ▷ emit the block of the step
  $\mathcal{C}_\theta \leftarrow \mathcal{C}_\theta[1{:}|s| - 1]$; $\mathcal{C}_q \leftarrow \mathcal{C}_q[1{:}\min(|\mathcal{C}_q|, |s| - 1)]$ ▷ both caches cropped to the new length -1
  $c \leftarrow c + r + 1$ ▷ upstream counter: counts the bonus token even when it was dropped
until $c \geq N$ **or** $\mathrm{last}(s) = e$
return s, $|s| - n$, $J = j$, $(\ell_1, \ldots, \ell_J)$

D PROMPTS FOR MODELS

The model works from three prompts, one per step, each printed on its own page. We give them here in a setup-independent form: we leave out file layout, tooling and algorithm-specific examples, so the prompts apply to any algorithm with a stated specification and can be adapted to another code base. The prompts are printed as plain text: their instructions on the typesetting of the documents are left out.

Each prompt covers one step of the method. Appendix D.1 writes the proof document of an algorithm, built mainly on its benchmarked code (Section 2.2). Appendix D.2 writes the Lean proof from a finished proof document. Appendix D.3 derives a variant from a parent's proof by changing its assumptions, with every change marked and every borrowed statement attributed. Figures 11 and 12 (Appendix A.2) show where each prompt is called and the workflow it sets out.

D.1 Designing proof files

Prompt 1: writing the proof document of a design

How to write a proof document: the code of one design as mathematics

A proof document takes one design (a method with its paper and the code that was benchmarked) and produces an exact mathematical description of that code, as it actually ran, stated only as assumptions, definitions, lemmas, theorems and proofs, ending in a theorem that running this code meets the design's specification: the one property the design is required to guarantee. That property is the only claim proved. Nothing about optimality, nothing about quality, nothing beyond it. The finished document is later formalized in Lean, following its proof structure.

The point of the whole exercise. The benchmarked code is the code for which we have performance numbers. The document is about that code: it shows that this code, when run, provably meets its specification, the way verified software is shown to meet a specification. Therefore the single most important requirement is that the document represents the code exactly, so that every definition is a faithful transcription of a line of the code that runs, and every design decision the code makes is explicit in the document. If the description is exact, a Lean proof about the description is a proof about the code. If it is not, the Lean proof certifies nothing. Everything else in this guide serves that requirement.

Inputs and outputs

Inputs, gathered for the design before a proof is started:

– the benchmarked code, with a hash identifying the exact version that ran;
– the paper;
– the authors' original paper source and original code, when available, with a record of where they came from (if either is missing, record that; do not pretend).

Outputs, stored with the design's other inputs:

– the proof document, structured as described under the heading on the proof document in order;
– a provenance record: the version of this guide the proof follows, the hashes of the proof document and of the code it was written against.

The proof is built mainly on the benchmarked code

The proof is built mainly on the benchmarked code. It is the code that produced the results; it is the ground source. Every assumption, definition, lemma and theorem of the document is about it. Before starting, confirm that the code at hand is the version that ran (its hash must match the version that produced the results). If it does not, stop and resolve that first; a proof about a different version is worthless.

The proof can also look at the paper and at the authors' reference code, to understand the benchmarked code:

– The authors' original code shows what a line of the benchmarked code is a port of and what an ambiguous line intends. Nothing from it enters the document unless the benchmarked code does the same thing, and then the document describes the benchmarked code.
– The paper and its source matter for one specific thing: assumptions. The paper motivates the method's steps and states the assumptions under which they are meant to work. Read it to understand the motivation of a part of the code and to find assumptions relevant to it (what is taken to be given, what is taken to be negligible, what a step presumes about its input). Such an assumption, when the code indeed relies on it, is written in the document as an assumption in the document's own words. Nothing is quoted from the paper, and no definition or theorem is written from the paper: the paper may tell you what to assume, the code tells you what is defined.

What is proved

For each design, one statement: running the benchmarked code on any input that satisfies the stated assumptions produces an output with the property the specification requires.

The specification is fixed by the problem the design solves, not by the design. State it precisely: what the input is,

what the output is, the quantity or relation the claim is about, and the exact form of the claim (an equality, an inequality, a containment, an identity of distributions). Where the design's output determines an exact amount (how much is saved, how many items are kept), prove the exact amount, not only its direction.

The problem statement is written so that the specification is the natural thing to prove: the problem states what must be achieved, the code is a procedure, and the theorem says the procedure achieves it. If the claim depends on how the output is stored or used, state that as an explicit assumption; without it the claim may not hold, and the document must say so in that assumption.

The proof document, in order

Everything in the document is one of: assumption, definition, lemma, theorem, proof. There are no explanatory paragraphs and no prose that describes the design in words, because such prose is not a statement that can be checked. What belongs is a definition as a formula, an assumption stating what is taken as given, and a theorem stating what follows. Every statement must be one that Lean could verify: it has a precise form (a formula, a set, a function, an inequality, an equality, a count), it names the assumptions it uses, and if it is a lemma or theorem it has a proof. Where a reader would need a word of explanation, put it inside the statement being explained (a definition's label, a remark line inside a definition saying which part of the code it transcribes), not in a paragraph between statements.

Symbols

First, list every symbol that appears in the definitions and in the theorems: shapes and sizes, inputs, parameters, intermediate objects, the quantity the specification is about. For each symbol give its meaning and where it is defined.

Problem statement

State what the input is, what the output is, the quantity or relation the specification is about, and the claim to be proved. Immediately after the problem statement: the assumptions the problem statement itself needs so that its objects exist, each a formal statement with quantifiers written out, and one lemma that the problem is well posed. All other assumptions are introduced at the section of the code that first needs them.

Where the problem definition comes from. If several designs should share one problem definition, the user provides it; use it as given, for every one of those designs. Otherwise, come up with the simplest problem definition that can be verified: the smallest input, output and claim that the code's behaviour can be checked against.

Reading the code exactly

Always, before writing a line of the proof, read the code itself. The document transcribes it directly. Every definition must follow the code:

– in the order the code runs, with the same indexing (say which convention is used and convert consistently), the same rounding, the same tie behaviour and the same boundary cases;
– covering every branch of the code that can run in the benchmark, including flags fixed to a constant. A branch that cannot run is handled by an assumption stating the setting that excludes it, as described under the heading on sections of the code;
– without programming–language constructs. Every library call, array operation or sentinel value becomes mathematics: a set–builder, a sum with its indices written out, a maximum over an explicit set, a selection with its tie–break, a slice with its bounds.

Sections of the code: assumptions, definitions, theorems

Divide the code into logical sections following its own structure: the main body in execution order and each function at the point where it is called. A section is one idea; it can be one line or several. If a single line needs small subsections to be defined properly, make the subsections. A line coverage list at the start assigns every line of the code that matters for the output (function headers, loops, returns, constant flags) to exactly one section; no line is skipped. Every line of the code is proved: it is defined, its assumptions are stated, and a theorem says what it establishes.

Each section consists only of:

1. Assumptions. What the section needs to be true, continuing the numbering. Each assumption is a formal statement with quantifiers written out, in method–independent terms (no line numbers, no document–internal symbols, no forward references), so it can be compared with other methods' assumptions. When a section reuses an earlier assumption, restate it in one line with its label. A benchmark setting that fixes a flag or excludes a branch of the code is an assumption here. An assumption the paper makes about a step and that the code relies on, as described under the heading on building the proof on the benchmarked code, is an assumption here too, in the document's own words.
2. Definitions. One definition per object the section computes, as a formula, with every choice the code makes spelled out (order, tie–break, boundaries, rounding). Each definition carries, inside it, the part of the code it transcribes, so that the correspondence from code to mathematics is itself part of the record. Where the code computes a thing in a particular way, the definition is the code's way, and a lemma states what it equals.
3. Lemmas and theorems, with proofs. From the section's definitions and assumptions: well–definedness, counts, ranges, partition facts, and always a specification clause: what the section does to the property the specification is about (preserves it, establishes it, or changes it by how much).

Every logic line of the code is a definition

Each line of the code that does something for the design's purpose is stated explicitly as a definition (or as an assumption, when it is a setting). Administrative lines, those that initialise components, register callbacks, log, reset counters or move data between devices, are not stated; everything else is, down to initial values, shapes, boundary cases, reductions and the exact form of the output.

A code coverage list names every function, branch and non–trivial statement of the code with the definition, assumption or theorem that states it, and for the administrative lines the word administrative. Nothing is too small. Each entry names the part of the code, the document object, and one of: defined, assumed (the setting that excludes or fixes it), or administrative (cannot affect the output). An entry that could affect the output and has no definition is a gap in the proof and must be closed before the document is finished.

Run the coverage test. When the document is drafted, and again after any change to it, go through the code line by line and check that every line is proved somewhere: each line of the code maps to a definition, an assumption or a theorem of the document, or is marked administrative with a reason. Check it in both directions: every entry of the coverage lists points to an existing statement, and every logic line of the code appears in an entry. The test passes only when no line is left uncovered; a single uncovered line means the proof does not describe the code, and the document is not finished.

The purpose is that every design decision in the code is explicit in the document, that every concept introduced in the code is covered by the mathematical notation, and that the covered pieces connect: each definition feeds the next, and the main theorem is reached by chaining them.

The main theorem: the code meets its specification

Combine the section theorems into the claim of the problem statement. The proof of the main theorem is pointers to the section theorems only. State explicitly every condition the claim needs (for strictness, for the output to be well defined) and where each comes from (assumption or lemma).

Index of assumptions

At the end: number, title, where stated (problem statement or section name), short form. By section name, never by line number.

Rules on sources

– Build mainly on the benchmarked code. Every definition is a transcription of a statement of the code. Use the paper and the authors' original code to understand the code, not to write the document.
– No quotations from the paper. Nothing from the paper's text, formulas or numbers appears in the document. The paper is read for motivation and for assumptions: when it states or implies an assumption behind a step the code performs, that assumption is written as an assumption in the document's own words, at the section that needs it, and used like any other assumption. Every definition and theorem is derived from the code and the assumptions alone.

Workflow for one design

1. Confirm the code is the version that ran. Read it fully. Read the matching parts of the original code and the method section of the paper only as far as needed to understand it.
2. Divide the code into sections, as described under the heading on sections of the code, and list its logic lines for the coverage lists.
3. Write the proof document in the order given above: only assumptions, definitions, lemmas, theorems, proofs.
4. Re−check the code. Read it again from top to bottom, with the finished document next to it, and confirm that every definition, assumption and theorem describes the code exactly: same branches, same indexing, same rounding, same ties, same boundary cases, every logic line with a definition and nothing in a definition that the code does not do. Walk the code coverage list entry by entry against the code. Fix the document where they differ.
5. Run the coverage test: every line of the code must be proved somewhere in the document. The document is finished only when it passes with no uncovered line.
6. Write the provenance record, as described under the heading on inputs and outputs.
7. Verification. Once everything is done, submit the proof document and its coverage lists to verification to catch any errors. Fix every error it reports and submit again; repeat until no errors remain, at most 5 rounds. Report any error still open after the fifth round.

Checklist

– The code is the version that ran: its hash matches the version that produced the results.
– Every line of the code is covered by the proof: the coverage test passes with no uncovered line.
– Every definition transcribes the code exactly, not the paper: same branches, indexing, rounding, ties and boundary cases.
– The proof answers the problem: the main theorem proves the specification of the problem statement.
– Every condition the main theorem needs is a stated assumption or a proved lemma; nothing is assumed silently.

D.2 Designing Lean from proof files

Prompt 2: writing the Lean proof from a proof document

How to write the Lean proof of a design from its proof document

The Lean proof formalizes a finished proof document. It shows that the design as implemented meets its specification . Its main theorem is a statement about the document's definitions, and the document's definitions are transcriptions of the benchmarked code, so the theorem is a statement about the code.

Inputs and outputs

Inputs:
– the finished proof document of the design, whose coverage test passes with no uncovered line;
– the benchmarked code the document was written against, with its hash.

Outputs, stored with the proof document:
– the Lean file;
– the provenance record of the proof document, extended with the hash of the Lean file.

Before writing any Lean

Re−check the code against the document. Read the code again from top to bottom, with the document next to it, and confirm that every definition, assumption and theorem describes the code exactly: same branches, same indexing, same rounding, same ties, same boundary cases, every logic line with a definition and nothing in a definition that the code does not do. Walk the list that assigns every line of the code to a statement, entry by entry, against the code. If they differ, fix the document first and run its coverage test again. Lean is only started once the document and the code match.

The Lean file follows the document

The Lean file follows the document's proof structure as closely as possible:
– one namespace per design, and a comment naming each document section, in document order;
– the problem's assumptions are the fields of a structure; later assumptions are fields of further structures introduced at the section that needs them, with the same labels in their docstrings;
– every definition of the document is a definition with the document's label in its docstring; every section theorem is a theorem with the same name and statement, including its specification clause; counts are their own lemmas;
– the main theorem is the statement of the document's main theorem, proved from the section theorems in the same order as the document;
– the main theorem depends only on Lean's standard foundations (propext, Quot.sound, Classical.choice); no sorry placeholder anywhere;
– indexing: say at the top which convention the Lean file uses and how document positions map to it.

The Lean file proves the same statement as the document, with the same intermediate lemmas in the same order.

Why the chain matters

A reader who trusts the Lean proof must be able to trust the code, because every object of the Lean file is a definition of the document, and every definition of the document is a transcription of a named part of the code. If that chain breaks anywhere (a definition that paraphrases the paper rather than the code, a logic line of the code with no definition), the Lean proof does not say anything about the code, and the document must not claim that it does.

Workflow

1. Confirm the document's coverage test passes and re−check the code against the document, as described under the heading on what to do before writing any Lean.
2. Write the Lean file following the document, as described under the heading on how the Lean file follows the document.
3. Compile it. Check what the main theorem depends on: only Lean's standard foundations, no sorry placeholder.
4. Add the hash of the Lean file to the provenance record.

5. Verification. Once everything is done, submit the Lean file to verification to catch any errors. Fix every error it reports and submit again; repeat until no errors remain, at most 5 rounds. Report any error still open after the fifth round.

Checklist

– The Lean file covers the code and the original proof document: every definition and theorem of the document is in it, under the same label.
– Lean compiles with no errors, internal errors included; no sorry placeholder.

D.3 Designing variants

Prompt 3: deriving a variant by changing one assumption

How to write a variant proof: perturb the assumptions of a design, derive a new design, prove its specification

A variant proof starts from a finished design proof (the parent) and produces the design proof of a new design (the child). The child's proof is the parent's proof rewritten:

– every unchanged statement is copied verbatim;
– every change is marked so that a reader can tell what was removed and what was added, with the new text written right after the old text it replaces;
– everything taken from another method is also marked as borrowed, with its source named.

The claim proved is the same: running the child's code meets the parent's specification. Always use the parent's problem definition: the child's problem statement is the parent's, copied verbatim.

Always start from the parent's code and proof document. Before anything else:

1. Create a new workspace for the child, named after the parent and the variant.
2. Copy the parent's code and proof document into it unchanged.
3. Make every change on these copies. The originals stay untouched: they give the parent's hash, the impact set and the final diff.

The principle: new designs come from perturbing assumptions

The parent's proof makes every assumption of the design explicit. That list is the parent's assumption set. Ideally, a new design is made by changing that set and deriving the algorithm it calls for:

1. Relax an assumption: weaken it, widen its bound, or drop it.
2. Augment the system: strengthen an assumption, narrow its bound, or add a new one.
3. Re–derive. Work out which definitions no longer follow from the assumptions, which new definitions the changed assumption requires or allows, and what the code must do differently. That code is the new design.

The direction is from assumptions to code, never from code to assumptions. A code tweak that changes no assumption is not a variant in this sense. If an idea starts from the code, first find the assumption it perturbs. If the parent's proof does not state that assumption (because the design choice is built into a definition), write it as an assumption first, then perturb it.

The best source of perturbations: what other methods assume

Methods for the same problem answer the same design questions differently. The answers are the assumptions of their proofs. They are already collected in the assumption catalog that comes with the design proofs: every assumption of every design proof, grouped by the question it answers (its class), with each method's pseudocode. Reuse this catalog; do not collect the assumptions again. Look through it for design features to reuse: an assumption, a definition, a rule, a pseudocode line.

State every borrowing. If the child uses an assumption or design feature of another method, name the source (method and label) in the perturbation summary and at the statement itself, and mark the borrowed text as borrowed in the child's proof.

To create a new design:

1. Pick a class in which the parent's choice differs from what other methods assume.
2. Replace the parent's assumption by another method's variant, or by a combination of the two (both, a mixture, an interpolation).
3. Derive the new design from the changed assumption set.

A technique that works in one method is then tried inside another design whose other assumptions stay fixed. This is how ideas move between methods in a controlled way: one assumption at a time.

Before borrowing, check the interface: the borrowed assumption must make sense with the parent's other assumptions (same objects, comparable units). If it does not, the derivation needs a compatibility step, marked as a change: a change of units, a renormalisation, or an extra assumption.

Kinds of perturbation

The moves below are examples, not a complete list. You can also use a perturbation that fits none of them, or a design feature that no other method has, if it is reasonable for the parent's design. It still has to change an assumption and be a good variant in the sense below. Such a feature is the child's own, so mark it as a change, not as borrowed.

– Borrow: replace the parent's variant of a class by another method's variant.
– Combine: keep the parent's variant and add another method's.
– Relax a bound: widen a range, or move to its endpoint.
– Augment a bound: narrow a range, or impose a floor.
– Drop: remove an assumption the code relies on.
– Add: a new requirement the design must satisfy.
– Split: a constant with two roles becomes two assumptions; perturb one.
– De–uniformise: one constant for all components becomes one per component.
– Change a quantifier: "for all" becomes "for a subset"; one aggregation becomes another.
– Interpolate: a parameter moving between two methods' variants of one class.

Controls (random choices, inverted decisions) are not new designs. They are reference points and make effect sizes interpretable.

What makes a good variant

1. One perturbation. One assumption is relaxed, augmented or borrowed, and every other edit follows from it. Two independent perturbations make two new designs. Compose them afterwards if both are interesting.
2. The change is real. The perturbed assumption changes what the code executes in the benchmark. The child is not equivalent to the parent: prove a separation lemma, a concrete input on which the two outputs differ. Before spending GPU time, measure how often the two outputs differ on a few inputs.
3. The parent is a special case. Prefer a family with a parameter that has a value which recovers the parent. Prove a reduction lemma: the child at that parameter value equals the parent. The lemma doubles as the check of the implementation, and sweeping the parameter gives a dose–response curve.
4. Small proof delta. A small perturbation touches one or two sections of the proof. If changes spread everywhere, the new design is far from the parent and no longer isolates one assumption.
5. Hypothesis from the proof, before the run. State what guarantee the perturbation removes or adds, and what that predicts.
6. Let the proof reveal confounds. Compute the impact set first. It shows when one assumption plays two roles, or when a perturbation silently triggers another theorem.
7. All else equal. Same hyperparameters, harness, samples, settings and models as the parent. Report the change against the parent per setting, not only the mean.
8. Not a duplicate. The same (parent, perturbation) pair has not been built already. The finished child is not one–to–one an existing method; otherwise it reuses that method's code and is not new, so it is not a variant.

Marking changes

– Mark every change so that a reader can tell what was removed, what was added and what was borrowed, and name the source of anything borrowed.
– Unchanged statements are verbatim: no rewording.
– Granularity: change the smallest unit (one bound, one symbol, one table cell, one pseudocode line, one code line). If more than half of a statement changes, mark the whole statement as removed and restate it.
– A removed statement stays in place, marked as removed.
– Assumption labels stay the parent's. A replaced assumption keeps the same number, with the old text marked as removed and the new text added. A changed bound keeps the same number, with only the bound edited. A removed assumption's number is retired, never reused. An added assumption gets a suffix label (for example a letter after the number) and is placed where first needed. Never renumber.
– An assumption, definition or pseudocode line taken from another method is new text and is also marked as

borrowed, with its source (method and label) named first. A borrowed assumption keeps the child's label. Its title also names the class it comes from. Its symbols are rewritten in the parent's notation.

The document, in order

1. Title: from the parent to the child, naming the changed assumptions, the changed lines of the code, and the proof that the child meets the specification. Then a short note that tells the reader how removed, added and borrowed text is marked.
2. Perturbation summary (facts only): the parent proof and its hash; the perturbed assumption, old and new, with its class, and, if it is borrowed, the method and label it comes from; the code lines changed and the hash of the child's code; the impact set; the guarantees lost and gained.
3. Symbols and problem statement: the problem statement is the parent's, verbatim; edits in the symbols only where the perturbation reaches them.
4. Pseudocode, checked against the child's code: a changed line is marked as removed, then the replacement follows as an unnumbered added line; an added line is tagged with the previous number and a letter; a removed line is marked as removed and keeps its number. The parent's line numbers stay valid.
5. The code change: the touched lines of the code in file order, each marked as removed or added, ending with "All other lines are unchanged". Then the list that assigns every line of the code to a statement and the list that assigns every statement to the code it covers, with the changed entries marked.
6. Sections, in the parent's order. For each statement, the treatment depends on its status. Outside the impact set: copied verbatim. Hypothesis list changes: edit the list and re−check the proof. Conclusion changes: edit the conclusion and the changed proof steps, and re−prove in full. Guarantee lost: mark the clause as removed and add a weaker version with its extra hypothesis, if one holds. Guarantee gained: add a new lemma or theorem. Every section theorem keeps its specification clause. If the child computes new quantities, extend the assumptions that cover the parent's quantities to cover them.
7. Main theorem for the child, then a short section "the child versus the parent": the reduction lemma; the separation lemma; the proof delta list: every parent statement, by label and title, with its status.
8. Index of assumptions: changed, removed and added entries marked as such.

The impact set

Before editing, compute on the parent's clean proof document, for the perturbed assumption (and, if needed, a symbol it introduces):

1. the statements that cite the perturbed assumption (rewrite candidates);
2. the statements downstream of them (re−check what they use);
3. the statements that name it only inside a hypothesis range;
4. the table and index lines that mention it.

Only these statements can change. Everything else is copied verbatim. The list is a reading aid, not a proof: the final re−read against the child's code is still required.

Workflow

1. Pick the parent. List its assumption set, adding the implicit design choices as assumptions.
2. Open the assumption catalog. For each class, compare the parent's choice with the other methods' variants. Choose one perturbation: borrow, combine, relax, augment, drop or add, or one of your own.
3. Create the child's workspace and copy the parent's code and proof document into it. All later steps edit these copies.
4. Write the perturbed assumption, the hypothesis and the expected guarantees lost or gained, before any code. Name the source method and label of anything borrowed.
5. Derive the new definitions. Implement them in the copied code with the smallest diff to the parent's. Record the hash and the diff.
6. Duplicate check, once the design is finished. Compare the child's assumptions and pseudocode with every known method. Look hardest at the method it borrows from and at the parent's family. There must be no one−to−one match. If one method could be identical, compare its outputs with the child's on a few inputs. Identical outputs mean the child is that method, not a variant: change the perturbation.
7. Before the GPU: the reduction check (the child at the parameter value that recovers the parent reproduces the parent's output) and the non−equivalence check (the outputs differ).
8. Compute the impact set on the parent's original proof document. Apply the edits to the copied document in

document order, marking each as removed, added or borrowed.
9. Re−read the child's code top to bottom against the finished document.
10. Diff the child's document against the parent's: every difference must be a marked change (removed, added or borrowed).
11. Run on the parent's evaluation setting. Report the change against the parent per setting.
12. Literature review, once the design is finished. Search published papers, preprints and open−source code for methods with the same assumptions or the same algorithm. List the closest ones with citations and say how the child differs from each. If a published method matches the child one−to−one, the design is not novel: change the perturbation.
13. Verification. Once everything is done, submit the child's code, proof document and literature review to verification to catch any errors. Fix every error it reports and submit again; repeat until no errors remain, at most 5 rounds. Report any error still open after the fifth round.

Checklist

– Child workspace created; started from copies of the parent's code and proof document.
– One assumption relaxed, augmented or borrowed; every other edit follows from it.
– Everything borrowed from another method: source method and label named, text marked as borrowed, interface checked.
– The finished child checked against every known method: no one−to−one match.
– The perturbation changes what the code executes; separation lemma proved.
– Reduction lemma, if the child is a family.
– Impact set computed; everything outside it verbatim, everything inside it re−checked.
– Removed text and added text marked as such; assumption labels stable.
– Specification clauses re−confirmed; main theorem proved for the child.
– Guarantees lost and gained listed; hypothesis written before the run.

E OpenEvolve and AIDE as run here

OpenEvolve (Sharma, 2025), an open implementation of AlphaEvolve (Novikov et al., 2025), evolves a population of programs in a MAP-Elites archive; AIDE (Jiang et al., 2025) is a tree search over whole programs that drafts, improves or debugs one node per step. Both run unmodified from pinned releases (OpenEvolve v0.4.0, commit 162c3fe; AIDE v0.2.2, commit ad73d22), start from the same parent code as our harness, and use the same model (Claude Fable 5.1, high effort, through an OpenAI-compatible proxy), the same two-stage evaluator (CPU checks on tiny random models, then the 12 protocol cells on the GPU queue) and the same budget: ten GPU-evaluated algorithms or 25 model calls.

Shared settings. Settings not listed below are the release defaults. Both tools call Claude Fable 5.1 (effort high) with no temperature, a reply cap of 100,000 tokens and a 3,600 s call timeout, and give the evaluator a 36,000 s timeout, since one evaluation waits for 12 queued GPU jobs. Each search stops at ten GPU-evaluated algorithms or 25 model calls, and we repeat it with three seeds per parent. None of these changes tunes a tool for a better score; each adapts it to the shared protocol.

OpenEvolve. OpenEvolve runs diff-based evolution with evolve blocks enforced (`enforce_evolve_blocks = true`), so edits outside the evolve block are reverted, which closes a reward hack that CacheCraft reports (Poudel et al., 2026). We shrink the population to fit ten algorithms: one island with a population of 20 and an archive of 10, no migration (`migration_rate = 0`), and five MAP-Elites feature bins, since the default five islands of 1,000 programs would never fill in ten algorithms. The elite, exploration and exploitation ratios are 0.2, 0.2 and 0.7; the prompt shows the three top programs and no diverse programs.

OpenEvolve evaluates the algorithms one at a time, so each algorithm sees all earlier ones. With `parallel_evaluations = 1` and a single cascade threshold of 0.5, algorithm k sees algorithms $1, \ldots, k-1$ and their per-cell score tables, which needs an artifact limit of 40,960 bytes. The code-length limit is 80,000 characters, since the parents have 400–650 lines.

AIDE. AIDE starts from the parent, with Fable as the coding model for drafting, improving and debugging. We insert the parent as the root node with its measured score (`num_drafts = 1`) instead of five drafts from scratch, so AIDE starts where the other searches start. Code formatting is off, so the parent's file is not rewritten, and every program runs in our two-stage evaluator instead of AIDE's interpreter.

AIDE takes its metric only from our evaluator. There is no review or report model: the evaluator's score is the metric, so no model call can report a metric that was not measured, and the metric is maximised (`metric_maximize = true`) rather than inferred. The step limit is 25 with the same stop at ten GPU-evaluated algorithms; `debug_prob` and `max_debug_depth` keep their defaults of 0.5 and 3.

F ALL ALGORITHMS

We formalized 89 algorithms, each with one proof graph: 44 for KV-cache eviction, 26 for speculative decoding and 19 for LLM weight pruning. The count is of proof graphs. It includes published configurations that combine or modify published methods (such as CaM on top of TOVA or LAMP on top of Wanda) and four standard baselines: autoregressive decoding, speculative sampling in two draft-length settings and the dense model. It leaves out the two reference points of KV-cache eviction, no compression and random eviction. The 44 KV-cache eviction algorithms are 30 KVPress algorithms, among them H2O (Zhang et al., 2023), SnapKV (Li et al., 2024a), PyramidKV (Cai et al., 2025), KNorm (Devoto et al., 2024), CaM (Zhang et al., 2024b), AdaKV (Feng et al., 2025), Expected Attention (Devoto et al., 2025), KeyDiff (Park et al., 2025), CUR (Sengupta et al., 2025) and CAP (Yang et al., 2026), and 14 ported algorithms, among them EchoPress (Lin et al., 2026) and DefensiveKV (Feng et al., 2026). The 26 speculative-decoding algorithms are 5 Spec-Bench algorithms (Lookahead (Fu et al., 2024), PLD (Saxena, 2023), REST (He et al., 2024), SAM-Decoding (Hu et al., 2025b) and token recycling (Luo et al., 2025)), 18 ported implementations and 3 baselines.

The 19 LLM weight-pruning algorithms are the dense model and 18 published algorithms and published settings (such as DSnoT without skipping, RIA with channel permutation and LAMP on top of Wanda or SparseGPT), among them magnitude pruning (Han et al., 2015), LAMP (Lee et al., 2021), SparseGPT (Frantar & Alistarh, 2023), Wanda (Sun et al., 2024), OWL (Yin et al., 2024), DSnoT (Zhang et al., 2024c), E-Sparse (Li et al., 2023), GBLM-Pruner (Das et al., 2023), RIA (Zhang et al., 2024a), Pruner-Zero (Dong et al., 2024), ALPS (Meng et al., 2024) and Z-Pruner (Bhuiyan et al., 2025). Figure 4 draws three algorithms of each field.

G UNCERTAINTY ESTIMATION

The error bars of the paper come from three sources of variation, and each figure uses the one that varies between its runs. A KV-cache eviction score is deterministic for a given set of questions (greedy decoding, fixed seed), so its uncertainty comes from which questions were drawn. A speculative-decoding speedup is a ratio of wall-clock times, so its uncertainty comes from repeating the same runs. A search result depends on the search's own random choices and on the model's replies, so its uncertainty comes from repeating the search. Unless stated otherwise, $\pm$ and error bars are one standard deviation (SD), and tests are two-sided, made cell by cell at $p < 0.05$ and not corrected for the number of cells.

G.1 KV-cache eviction: resampling the questions

The SD of a KV-cache eviction score comes from a bootstrap over its questions. A RULER score is the mean over the 13 tasks of the per-task accuracy on the drawn questions (LongBench: 16 tasks). We resample the questions of each task with replacement, keeping each task's count, recompute the score, and repeat this 10,000 times; the SD of the 10,000 scores is the bootstrap SD reported next to every score. It says how much the score would move under another draw of the same number of questions, and it shrinks with that number: 0.2 to 0.5 points per cell at 6,500 RULER-4K questions.

Means over cells and differences between two algorithms combine the per-cell SDs. A mean over cells (the 12 model–factor cells, the two models, or the six factors) treats the cells as independent runs, so the SD of a mean of k cells is $\sqrt{\sum_i \mathrm{sd}_i^2}/k$. A difference between two algorithms has SD $\sqrt{\mathrm{sd}_a^2 + \mathrm{sd}_b^2}$ and we test it with a z-test; this ignores that both algorithms answer the same questions, which usually makes the test conservative. The mutation study pairs the two instead: it scores the mutant and its parent on the same resampled questions, and the 2.5th and 97.5th percentiles of the 10,000 resampled differences give a 95% interval.

G.2 Speculative decoding and systems profiles: repeating the runs

Each Spec-Bench cell runs three times with seed 0, and its error bar is the SD over these runs. We submit run k of the parent and run k of the variant back to back to the same machine, and each speedup divides by the autoregressive run of the same model and temperature. A cell reports the mean of the three runs and, as its error bar, their sample SD; we test a difference with a paired t-test over the three run pairs (two degrees of freedom). This SD measures timing variation (clock speeds, scheduling, other jobs on the machine), not variation over questions: every run uses the same 480 questions. Accepted tokens per target pass (#MAT) are identical in every run at a fixed seed, so they carry no error bar and their differences are exact.

The systems profiles measure variation over questions instead, with a paired bootstrap. The profiles behind panels G and I of Figure 7 run three paired timing runs per cell on all 480 Spec-Bench questions; the error bars of panel G are the SD over the three runs, those of panel I the SD of a paired question bootstrap (10,000 resamples of the question ids, the same resample applied to every run and to the paired autoregressive passes). Table 8 and the panel C of Figure 7 use seven passes over two fixed prompts; their error bars are the sample SD over the passes.

G.3 Design searches: repeating the search

Each harness runs three times from the same parent. OpenEvolve and AIDE take a different seed in each repeat. Our harness has no seed: the repeat number only labels the run, and the model's proposals are not seeded, so its repeats largely replay the same sequence of moves. Figure 8 plots, after each algorithm, the mean over the three repeats of the running-best score, with error bars spanning the minimum and maximum over the repeats; the text also reports the sample SD over the repeats.

The min–max range and the SD are rough estimates, and both understate the variation of our harness. Both contain the search's own variation and the question-draw uncertainty of each score (about 0.45 points for a 12-cell mean at 200 questions). With three repeats, each is itself a rough estimate, and for our harness, whose repeats are not independent, each understates the variation one would see across independent searches.

H THE WORKED EXAMPLE IN FULL

Section 2.1 sketched why speculative sampling (Leviathan et al., 2023) is exact; this appendix walks through the whole argument. A small drafter proposes tokens, the large target model checks them in one pass, and the text that comes out must be distributed exactly as if the target had written it alone. We first fix the setting and the problem, then follow one drafted token through the accept-or-resample step, show that the token that comes out has the target's law, extend this to a block of drafts and to the whole loop, and end with how many tokens one target pass buys.

Every statement carries the name it has in Figure 3, whose graph is repeated below; that figure shows Assumption $\mathcal{A}_5$, Definition $\mathcal{D}_3$, Theorem $\mathcal{T}_1$ and the problem $\mathcal{P}$ word for word. Every lemma, theorem and the corollary below is proved in Lean 4 with Mathlib, statement for statement (no `sorry`, no added unproved statement).

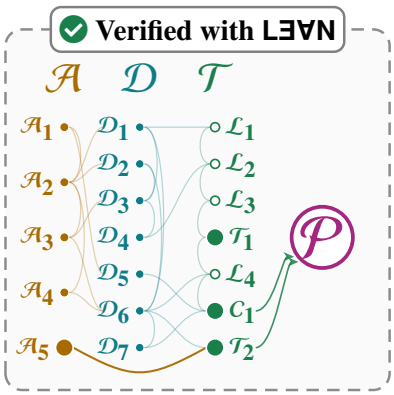

Figure 3. **The proof graph of this appendix, repeated from Figure 3.** Lines are drawn after transitive reduction, so indirect citations are omitted; the arrows into $\mathcal{P}$ come from the claims that answer the problem. Enlarged discs mark Assumption $\mathcal{A}_5$, which SVIP relaxes, and the exactness and token-count claims; the dark curve along the bottom is the only citation of Assumption $\mathcal{A}_5$, by Theorem $\mathcal{T}_2$.

In the repeated graph (Figure 3), each disc is the statement of the same name below; a line from one disc to another means the second cites the first.

H.1 The setting and the problem

Two language models share a finite vocabulary $\mathcal{V}$. We write $\mathcal{V}^*$ for the finite sequences over it, $u \in \mathcal{V}^*$ for a prefix, and call a map $\mathcal{V} \to [0,1]$ that sums to 1 a *distribution* on $\mathcal{V}$. The first assumption says what the large model is: something that, given a prefix, returns a distribution over the next token. Where it matters, an assumption also says what it does *not* require, because that is what the agent is allowed to change.

Assumption $\mathcal{A}_1$ (Target distribution over a finite vocabulary)**.** For every prefix $u \in \mathcal{V}^*$ the target model M_p defines a distribution $p(\cdot \mid u)$ on $\mathcal{V}$. Temperature, top-k and nucleus adjustments are folded into p (standardized sampling); temperature 0 makes $p(\cdot \mid u)$ a point mass on an arg max (ties broken by a fixed rule). Nothing is assumed about how p depends on u.

The second model is the drafter. It must use the same vocabulary, and nothing else is asked of it: the argument has to work however poor the drafter is.

Assumption $\mathcal{A}_2$ (Draft distribution over the same vocabulary)**.** For every prefix u the draft model M_q defines a distribution $q(\cdot \mid u)$ on the same $\mathcal{V}$ as Assumption $\mathcal{A}_1$. Nothing relates q to p: q need not be close to p, need not have $q(x \mid u) > 0$ where $p(x \mid u) > 0$, and may be a point mass.

With the two models fixed, the problem can be stated.

The problem $\mathcal{P}$**.** Given a target model M_p and a cheaper drafter M_q over the same vocabulary, emit a continuation whose law is exactly that of autoregressive sampling from M_p (Corollary $\mathcal{C}_1$), while one target call emits up to $\gamma + 1$ tokens, with the expected number given by Theorem $\mathcal{T}_2$.

H.2 One position: accept or resample

Speculative sampling decides the fate of one drafted token at a time. Fix one position and write p, q for the target's and the drafter's distributions there. A token is accepted with certainty when the target likes it at least as much as the drafter, and otherwise only in proportion:

Definition $\mathcal{D}_1$ (Acceptance probability)**.** For p and q at one position (Assumptions $\mathcal{A}_1$ and $\mathcal{A}_2$), $a(x) = \min\big(1,\, p(x)/q(x)\big)$ when $q(x) > 0$, and $a(x) = 1$ when $q(x) = 0$. (The value at $q(x) = 0$ never matters: such an x is never drawn from q. A Lean file is free to use its own convention for division by zero; Lemma $\mathcal{L}_1$ holds for every choice.)

A rejected token is not replaced by a fresh draw from p, which would give the accepted tokens too much weight. The replacement comes from the part of p that acceptance leaves uncovered:

Definition $\mathcal{D}_2$ (Residual distribution)**.** For p and q at one position (Assumptions $\mathcal{A}_1$ and $\mathcal{A}_2$) let $Z = \sum_x \max(0,\, p(x) - q(x))$. If $p \neq q$ then $Z > 0$ (if $Z = 0$ then $p \leq q$ everywhere, and as both sum to 1, $p = q$), and $p'(x) = \max(0,\, p(x) - q(x))/Z$. For $p = q$ set $p' = p$; by Lemma $\mathcal{L}_3$ the branch that draws from p' then has probability 0, so any distribution would do.

The step also needs its randomness to behave: drafts drawn from q, an acceptance coin drawn independently of them, and a replacement drawn from the residual. The assumption is stated for a step of γ drafts, used later; a single position is its first draft.

Assumption $\mathcal{A}_3$ (Independent verifier randomness)**.** Within a step at prefix u the random draws are the draft tokens $x_1, \ldots, x_\gamma$, the verifier's $r_1, \ldots, r_\gamma$ and one final token x' (the residual token after a rejection, otherwise the token appended after a full acceptance). (i) Given everything drawn before it, x_i has law $q(\cdot \mid u\, x_{1:i-1})$ of Assumption $\mathcal{A}_2$. (ii) Each r_i is uniform on $[0, 1]$ and independent of $(x_{1:\gamma}, r_{1:\gamma} \setminus r_i)$. (iii) The residual token, given everything drawn before it, has the residual distribution of Definition $\mathcal{D}_2$ for the position at which it is drawn. (iv) The draws of a step are independent of the draws of earlier steps given the prefix at which the step starts.

Together these make one step:

Definition $\mathcal{D}_3$ (One speculative-sampling step at one position)**.** Draw $x \sim q$ (Assumption $\mathcal{A}_2$) and $r \sim U[0, 1]$ (Assumption $\mathcal{A}_3$). If $r \leq a(x)$ (Definition $\mathcal{D}_1$) the step *accepts* and emits x; otherwise it *rejects* and emits $x' \sim p'$ (Definition $\mathcal{D}_2$).

Two pointwise identities carry all the algebra that follows. The first says that a token is drafted and then accepted with probability $\min(p, q)$; the second that the residual weight of a token is what p has left beyond that minimum.

Lemma $\mathcal{L}_1$ (Pointwise identities)**.** *For every $x \in \mathcal{V}$, with a of Definition $\mathcal{D}_1$: (a) $q(x)\, a(x) = \min(p(x), q(x))$; (b) $\max(0,\, p(x) - q(x)) = p(x) - \min(p(x), q(x))$.*

Proof. (a) If $q(x) = 0$ both sides are 0, since $p(x) \geq 0$. If $q(x) > 0$ and $p(x) \geq q(x)$ then $a(x) = 1$ and both sides are $q(x)$. If $q(x) > 0$ and $p(x) < q(x)$ then $a(x) = p(x)/q(x)$ and both sides are $p(x)$. (b) If $p(x) \geq q(x)$ both sides are $p(x) - q(x)$; otherwise both are 0. □

How often does a step accept? Leviathan et al. (2023) name this rate and relate it to a divergence between the two models:

Definition $\mathcal{D}_4$ (Acceptance rate and divergence, Defs. 3.1 and 3.2 of Leviathan et al. (2023))**.** $\beta = \Pr(\text{accept})$ in Definition $\mathcal{D}_3$, and $D_{LK}(p, q) = \sum_x |p(x) - M(x)|$ with $M = \frac{1}{2}(p + q)$.

Summing the first identity over all tokens gives the rate in closed form, and the second identity shows that the residual is normalized by exactly the probability of a rejection:

Lemma $\mathcal{L}_2$ (Acceptance rate, normalizer, divergence; Lemma 3.3 and Theorem 3.5 of Leviathan et al. (2023))**.** *Under Assumption $\mathcal{A}_3$, the acceptance rate β and the divergence D_{LK} of Definition $\mathcal{D}_4$ and the normalizer Z of Definition $\mathcal{D}_2$ satisfy $\beta = \sum_x \min(p(x), q(x))$, $Z = 1 - \beta$, and $D_{LK}(p, q) = 1 - \beta$.*

Proof. By Assumption $\mathcal{A}_3$(ii), r is independent of x and uniform, so $\Pr(\text{accept} \mid x) = \Pr(r \leq a(x)) = a(x)$ because $a(x) \in [0, 1]$; hence $\beta = \sum_x q(x)\, a(x) = \sum_x \min(p(x), q(x))$ by Lemma $\mathcal{L}_1$(a). By Lemma $\mathcal{L}_1$(b), $Z = \sum_x \big(p(x) - \min(p(x), q(x))\big) = 1 - \beta$. Pointwise, $|p - M| = \frac{1}{2}|p - q| = \frac{1}{2}\big(\max(p, q) - \min(p, q)\big)$ and $\max(p, q) + \min(p, q) = p + q$, which sums to 2; so $D_{LK} = \frac{1}{2}\big(2 - 2\sum_x \min(p, q)\big) = 1 - \beta$. □

Finally, a step can only be sure to accept when the two models agree, which is the case that Definition $\mathcal{D}_2$ set aside:

Lemma $\mathcal{L}_3$ (When nothing is rejected)**.** $0 \le \beta \le 1$, *and* $\beta = 1$ *if and only if* $p = q$.

Proof. By Lemma $\mathcal{L}_2$, $\beta = \sum_x \min(p(x), q(x))$; $0 \le \min(p, q) \le p$ pointwise and $\sum_x p(x) = 1$, so $0 \le \beta \le 1$. If $\beta = 1$ then $\sum_x \big(p(x) - \min(p(x), q(x))\big) = 0$ with non-negative terms, so $\min(p(x), q(x)) = p(x)$, i.e. $p(x) \le q(x)$, for every x; if $p(x_0) < q(x_0)$ for some x_0 then $1 = \sum_x p(x) < \sum_x q(x) = 1$, a contradiction, so $p = q$. Conversely $p = q$ gives $\beta = \sum_x p(x) = 1$. □

H.3 The emitted token has the target's law

Everything is now in place for the central claim: whatever the drafter, the token that one step emits is distributed as p. The proof adds up the two ways a token can come out, accepted as a draft or drawn from the residual.

Theorem $\mathcal{T}_1$ (Exactness of one step; Leviathan et al. (2023))**.** *Under Assumptions* $\mathcal{A}_1$*–*$\mathcal{A}_3$*, the token emitted by Definition* $\mathcal{D}_3$ *has law* p*:* $\Pr(\textit{emit } y) = p(y)$ *for every* $y \in \mathcal{V}$.

Proof. The step emits y either by accepting a draft equal to y or by rejecting and drawing y from p'; the two events are disjoint, so

$$\Pr(\text{emit } y) = \Pr(\text{accept, } x = y) + \Pr(\text{reject, residual} = y).$$

Accept path. $\Pr(\text{accept, } x = y) = q(y)\Pr(r \le a(y) \mid x = y) = q(y)\,a(y) = \min(p(y), q(y))$, by Assumption $\mathcal{A}_3$(ii) and Lemma $\mathcal{L}_1$(a). *Reject path.* By Assumption $\mathcal{A}_3$(iii) the residual token, given (x, r), has law p', so $\Pr(\text{reject, residual} = y) = \Pr(\text{reject})\,p'(y) = (1 - \beta)\,p'(y)$. *Case* $p \ne q$. By Lemmas $\mathcal{L}_2$ and $\mathcal{L}_3$, $Z = 1 - \beta > 0$, so $(1 - \beta)\,p'(y) = \max(0, p(y) - q(y)) = p(y) - \min(p(y), q(y))$ by Lemma $\mathcal{L}_1$(b), and the two paths add to $p(y)$. *Case* $p = q$. By Lemma $\mathcal{L}_3$, $\beta = 1$, the reject path has probability 0, and the accept path gives $\min(p(y), p(y)) = p(y)$. □

One position is therefore exact. A real step, however, drafts several tokens before the target checks any of them.

H.4 From one token to a block, and to the whole loop

In the code the drafter writes γ tokens one after another, and the target then checks all of them in a single forward pass. The next assumption says what that pass returns and how its verdicts are used.

Assumption $\mathcal{A}_4$ (Token-wise verification)**.** At prefix u with drafts $x_{1:\gamma}$, the target forward pass returns the $\gamma+1$ distributions $p_i = p(\cdot \mid u\,x_{1:i-1})$, $i = 1, \dots, \gamma + 1$, of Assumption $\mathcal{A}_1$; drafted token x_i is verified against p_i with draft distribution $q_i = q(\cdot \mid u\,x_{1:i-1})$ of Assumption $\mathcal{A}_2$; the first rejection ends the step; if all γ drafts are accepted, one token is drawn from $p_{\gamma+1}$, given everything drawn before it, and appended.

Exactness is a claim about whole sequences, so we name the law that the target alone gives a sequence,

Definition $\mathcal{D}_5$ (Autoregressive reference)**.** For a prefix u and $L \ge 0$, $P_L^{\mathrm{AR}}(y_{1:L} \mid u) = \prod_{t=1}^{L} p(y_t \mid u\,y_{1:t-1})$ with p of Assumption $\mathcal{A}_1$, the law of L tokens sampled one at a time from M_p.

and the step exactly as the code runs it:

Definition $\mathcal{D}_6$ (A step of γ tokens, as the code runs it)**.** At prefix u: draw $x_1, \dots, x_\gamma$ in order, $x_i \sim q(\cdot \mid u\,x_{1:i-1})$ (Assumption $\mathcal{A}_2$; the drafter's autoregressive loop); obtain $p_1, \dots, p_{\gamma+1}$ from one target pass (Assumption $\mathcal{A}_4$); draw $r_1, \dots, r_\gamma$ (Assumption $\mathcal{A}_3$); let $n = \min\{\, i : r_i > a_i(x_i) \,\} - 1$, with $n = \gamma$ if there is no such i, where a_i is Definition $\mathcal{D}_1$ for (p_i, q_i). The step emits the block $y = x_{1:n}\,x'$, where $x' \sim p'_{n+1}$ (Definition $\mathcal{D}_2$ for p_{n+1} against q_{n+1}) if $n < \gamma$ and $x' \sim p_{\gamma+1}$ if $n = \gamma$. The block has $n + 1 \in \{1, \dots, \gamma + 1\}$ tokens.

Because all γ drafts are drawn before any of them is verified, it is not obvious that the emitted tokens still behave as fresh steps of Definition $\mathcal{D}_3$. The next lemma shows that they do; it is the only place where the order of the draws matters.

Lemma $\mathcal{L}_4$ (Token-level kernel of a step)**.** *Consider Definition* $\mathcal{D}_6$ *at prefix* u *under Assumptions* $\mathcal{A}_1$*–*$\mathcal{A}_4$*. Let* $1 \le i \le \gamma$*, let* $y_{1:i-1} \in \mathcal{V}^{i-1}$*, and let* $E_i = \{n \ge i - 1,\ x_{1:i-1} = y_{1:i-1}\}$ *be the event that the first* $i - 1$ *drafts equal* $y_{1:i-1}$ *and were all accepted. If* $\Pr(E_i) > 0$ *then, conditional on* E_i*, the* i*-th emitted token* y_i *has law* $p(\cdot \mid u\,y_{1:i-1})$*. For every* $y_{1:\gamma}$ *with* $\Pr(n = \gamma,\ x_{1:\gamma} = y_{1:\gamma}) > 0$*, conditional on this event the* $(\gamma + 1)$*-th emitted token has law* $p(\cdot \mid u\,y_{1:\gamma})$.

Proof. E_i is determined by $(x_{1:i-1}, r_{1:i-1})$. By Assumption $\mathcal{A}_3$(i), conditional on $x_{1:i-1} = y_{1:i-1}$ the draft x_i has law $q(\cdot \mid u\, y_{1:i-1})$, and conditioning further on $r_{1:i-1}$ does not change this, since by (ii) the r's are independent of the drafts; by (ii) again, r_i is uniform and independent of $(x_i, x_{1:i-1}, r_{1:i-1})$. So conditional on E_i the pair (x_i, r_i) has law $q(\cdot \mid u\, y_{1:i-1}) \otimes U[0,1]$. By Assumption $\mathcal{A}_4$, $p_i = p(\cdot \mid u\, y_{1:i-1})$ and $q_i = q(\cdot \mid u\, y_{1:i-1})$ on E_i. On E_i the step is still running at position i, and the i-th emitted token is x_i if $r_i \le a_i(x_i)$ and the residual draw of position i otherwise, which by (iii) has law p_i' given everything before it. This is exactly Definition $\mathcal{D}_3$ for (p_i, q_i), so Theorem $\mathcal{T}_1$ gives the law $p_i = p(\cdot \mid u\, y_{1:i-1})$. The last claim is Assumption $\mathcal{A}_4$: given everything drawn before it, the appended token has law $p_{\gamma+1} = p(\cdot \mid u\, y_{1:\gamma})$. □

Repeating the block step gives the loop,

Definition $\mathcal{D}_7$ (The speculative loop)**.** Starting from u, repeat Definition $\mathcal{D}_6$ at the current prefix and append the block, until at least L tokens have been emitted; $P_L^{\mathrm{SpS}}(\cdot \mid u)$ is the law of the first L emitted tokens $Y_{1:L}$.

and the loop is exact: its first L tokens have the law of L tokens sampled from the target one at a time.

Corollary $\mathcal{C}_1$ (Exactness of the loop)**.** *Under Assumptions $\mathcal{A}_1$–$\mathcal{A}_4$, $P_L^{\mathrm{SpS}}(\cdot \mid u) = P_L^{\mathrm{AR}}(\cdot \mid u)$ for every prefix u and every $L \ge 0$, with P_L^{SpS} of Definition $\mathcal{D}_7$ and P_L^{AR} of Definition $\mathcal{D}_5$.*

Proof. Let $Y_1, Y_2, \ldots$ be the emitted tokens of Definition $\mathcal{D}_7$ in order. We show that for every $1 \le t \le L$ and every $y_{1:t-1}$ with $\Pr(Y_{1:t-1} = y_{1:t-1}) > 0$,

$$\Pr(Y_t = y \mid Y_{1:t-1} = y_{1:t-1}) = p(y \mid u\, y_{1:t-1}) \qquad \text{for every } y \in \mathcal{V}; \tag{1}$$

the chain rule then gives $\Pr(Y_{1:L} = y_{1:L}) = \prod_{t=1}^{L} p(y_t \mid u\, y_{1:t-1}) = P_L^{\mathrm{AR}}(y_{1:L} \mid u)$.

Token t is emitted by some step; which step, and at which position $i \in \{1, \ldots, \gamma+1\}$ within it, is determined by the lengths of the blocks emitted before it. Let $H = (\ell_1, \ldots, \ell_k)$ range over the lengths of the blocks that end before token t ($\ell_1 + \cdots + \ell_k \le t-1$, the next block containing token t); the events $\{Y_{1:t-1} = y_{1:t-1}\} \cap H$ partition $\{Y_{1:t-1} = y_{1:t-1}\}$, so it suffices to prove (1) conditional on each such cell of positive probability. Fix H; it places token t at position $i = t - (\ell_1 + \cdots + \ell_k)$ of a step that started at prefix $w = u\, y_{1:t-i}$, and on the cell the first $i-1$ tokens of this step are $y_{t-i+1:t-1}$ and were accepted. By Assumption $\mathcal{A}_3$(iv), the draws of this step are independent of the earlier steps given w, so conditioning on the earlier steps (which fix w and H up to this step) changes nothing about this step beyond fixing w. If $i \le \gamma$ the cell is the event E_i of Lemma $\mathcal{L}_4$ for the step at w with $y_{1:i-1} := y_{t-i+1:t-1}$, so Y_t has law $p(\cdot \mid w\, y_{t-i+1:t-1}) = p(\cdot \mid u\, y_{1:t-1})$. If $i = \gamma+1$ the cell says $n = \gamma$ with $x_{1:\gamma} = y_{t-\gamma:t-1}$, and the last claim of Lemma $\mathcal{L}_4$ gives the same law. Hence (1) holds on every cell, and therefore on their union. □

The proof applies to the idealized loop defined above, with its stated conditional sampling and independence assumptions. It does not certify arbitrary stopping rules or the measured SVIP cap branch.

H.5 How many tokens one target pass buys

Exactness says nothing about speed. The speedup comes from emitting more than one token per target pass, and to count them Leviathan et al. (2023) idealize the acceptances:

Assumption $\mathcal{A}_5$ (Fixed draft length, independent acceptances)**.** $\gamma \ge 1$ is a constant, and in a step of Definition $\mathcal{D}_6$ the first i drafts are all accepted with probability α^i ($0 \le i \le \gamma$) for a rate $\alpha \in [0,1]$, as when acceptances are independent with common rate α, e.g. when β of Definition $\mathcal{D}_4$ equals α at every prefix. This is an idealization used only by the cost claim (Theorem $\mathcal{T}_2$); the exactness claims never cite it.

Under this idealization the count is a geometric sum.

Theorem $\mathcal{T}_2$ (Expected tokens per target pass; Equation (1) of Leviathan et al. (2023))**.** *Under Assumption $\mathcal{A}_5$, a step of Definition $\mathcal{D}_6$ emits $\mathbb{E}[n+1] = \sum_{i=0}^{\gamma} \alpha^i$ tokens, which is $(1-\alpha^{\gamma+1})/(1-\alpha)$ for $\alpha < 1$ and $\gamma+1$ for $\alpha = 1$.*

Proof. $n+1 = \sum_{i=0}^{\gamma} \mathbf{1}[\text{the first } i \text{ drafts are accepted}]$, since the indicator is 1 exactly for $i \le n$. Under Assumption $\mathcal{A}_5$ the i-th indicator has expectation α^i; sum the geometric series. □

Without Assumption $\mathcal{A}_5$, for a fixed maximum draft length the general tail-sum identity is $\mathbb{E}[n{+}1] = \sum_{i=0}^{\gamma} \Pr(A_1 \cap \cdots \cap A_i)$, with the empty intersection ($i = 0$) having probability one. Here A_i denotes acceptance at position i. Prefix-dependent rates cannot in general be multiplied as unconditional marginal probabilities; adaptive draft lengths additionally require accounting for whether a position was proposed.

I MUTATION STUDY

I.1 Exactness proofs of published decoders

Formalizing a field also checks the algorithms already in it. We prove exactness for eight published speculative decoders under stated cache, tie-breaking and exact-arithmetic assumptions; five of them claimed it without proof (Gante, 2023; Mamou et al., 2024b; Zhang et al., 2025; Agrawal et al., 2024).

I.2 Proof checks catch every injected bug, benchmarks only about half

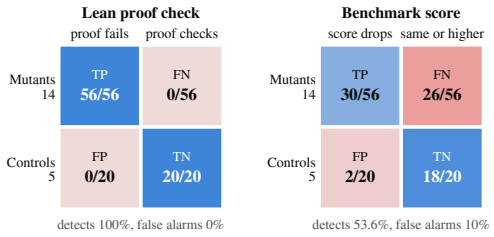

Figure 14. **Lean flags every mutant; the benchmark flags about half.** An evaluation is one design at one setting, Llama-3.1-8B-Instruct or Qwen3-8B at $16\times$ or $64\times$ compression, so each design counts four times. Lean gives one verdict per design, shared by its four settings; the benchmark flags any score below the parent's, the rule most favourable to it. Blue cells are correct decisions and red cells wrong ones, shaded by their share of the row.

We took five verified KV-cache eviction algorithms (KNorm, SnapKV, KVGrad, RestoreKV and KVzip) and wrote 14 code mutants. Each mutant injects one realistic bug, such as keeping one pair too many or too few, reversing a ranking, skipping a layer or forgetting to protect the sink tokens; comment-only edits and an equivalent rewrite serve as five controls. We checked each parent's fixed Lean proof against the mutated code and ran every mutant and control on 200 RULER-4K questions. Every mutant breaks its proof and every control passes, yet in 26 of the 56 mutated evaluations the score matches or exceeds the parent's: some bugs are too small to move the score, and others raise it by silently exceeding the memory budget, such as keeping every pair.